

极 简 教 材

看懂 Transformer 所需的线性代数

一个对象，一个运算的三种读法，
三种几何操作，两个不等式，两个数字。

刘玉书 著

十二章 · 四部 · 每章配 NumPy 实验与自测题
不含行列式 · 不含矩阵求逆 · 不含高斯消元 · 不含特征分解

目录

导读	3
第一部 表示 建立语言	
第 1 章 向量与嵌入空间	5
第 2 章 矩阵乘法的三种读法	11
第 3 章 形状代数	21
第二部 注意力 主线的高潮	
第 4 章 线性变换与投影	29
第 5 章 内积、范数与打分	37
第 6 章 加权平均与凸组合	45
第 7 章 秩与低秩瓶颈	52
第三部 几何 把模型看成空间中的运动	
第 8 章 加法、残差流与高维容量	60
第 9 章 归一化的几何	68
第 10 章 正交、旋转与位置	75
第四部 收束 拼回去	
第 11 章 升维、非线性与深度	83
第 12 章 从零拼出一层	91

导读

这本书只讲一件事：**理解 Transformer 需要哪些线性代数，以及为什么是这些。**

全书主线

整个 Transformer 可以概括成一句话——在一个 d 维向量空间里，反复做**线性变换 + 加权平均 + 一次非线性**。

所以真正需要掌握的内容，比通用线性代数教材少得多，但要求理解得更深。全书按“读懂模型时会卡在哪儿”组织，而不是按数学对象组织：每一章解决一个问题，不解决一个概念。

本书不讲什么

以下内容整块删除，不是因为它们不重要，而是因为它们服务的是别的问题——Transformer 的前向传播里没有解方程，也没有求逆：

行列式与克拉默法则、高斯消元与线性方程组求解、矩阵求逆与伴随矩阵、特征分解的一般理论、若尔当标准型、二次型分类、抽象向量空间的公理化定义、一般的基变换理论。

每章末尾都有一节“本章刻意没讲”，说明省略了什么、以及在哪里会以更具体的形式重新出现。读者应该知道自己被省掉了什么。

体例

每章固定六段：一个具体的困惑 → 最小必要定义 → 两到三种直觉 → 一段可运行的代码 → 明确落回 Transformer 的哪一行 → 自测题。代码只用 NumPy，直到最后一章——框架会隐藏形状，而形状恰恰是本书要训练的东西。

全书只保留三处正式推导： $\sqrt{d_k}$ 的方差计算（第 5 章）、 $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$ （第 7 章）、RoPE 的相对位置恒等式（第 10 章）。其余用图和数值例子。

读完之后应该能回答

1. 不查资料写出 $d=512$ 、 $h=8$ 的一层里每个张量的形状。
2. 用“行向量加权求和”解释 $A \cdot V$ ，说明输出为何落在 V 的凸包内。
3. 从方差角度推导 $\sqrt{d_k}$ 。
4. 说明 $W_q W_k^T$ 的秩上界，以及这对多头设计意味着什么。

5. 证明旋转后的内积只依赖相对位置。
6. 说明 LayerNorm 把向量映到了什么几何对象上。
7. 用 $12d^2$ 估算模型参数量。
8. 解释为什么去掉所有非线性后，96 层模型等价于 1 层。
9. 论证 768 维空间为什么能容纳远超 768 个概念，给出定量依据。
10. 写出 $\text{FFN}(x) = x W_1 D_x W_2$ ，并用它说明非线性带来了什么。

学习顺序建议

第 2 章（矩阵乘法的三种读法）是全书支点，值得反复读；第 3 章（形状代数）枯燥但必要。这两章练到不用思考，后面九章会顺得多。第三部（第 8~10 章）是全书唯一带审美性质的部分，也是读完之后最容易长期记住的部分。

第 1 章 向量与嵌入空间

本章要回答：一个词是怎么变成一串数字的？为什么是 768 个数，而不是 3 个或者 3 亿个？

读完本章你能做到：解释嵌入矩阵的形状与参数量；说清"坐标"和"方向"哪个才携带语义；用余弦相似度做第一个数值实验。

1.1 困惑：从符号到数

计算机只会做算术。要让模型处理 "the cat sat"，第一步必须把每个 token 换成数。

最朴素的办法是编号：the=0, cat=1, sat=2。这立刻出问题——编号带来了虚假的顺序和距离。按这个方案，cat 和 sat 相差 1，cat 和 the 也相差 1，而 token 5000 和 token 5001 会被认为极其相似。可它们只是恰好在词表里挨着。

第二种办法是 one-hot 编码：词表有 V 个词，就用 V 维向量表示，第 i 个词是第 i 位为 1、其余全 0 的向量。

```
the      = (1, 0, 0, 0, ..., 0)    ← 50257 维
cat      = (0, 1, 0, 0, ..., 0)
democracy = (0, 0, 0, 1, ..., 0)
```

这解决了虚假距离的问题，但代价过高：

- **维度爆炸**：GPT-2 的词表有 50257 个 token，每个词占 50257 个数，其中 50256 个是零。
- **距离全都一样**：任意两个 one-hot 向量的内积都是 0。cat 与 dog 的相似度，和 cat 与 democracy 的相似度，在数学上完全相等，都是零。

第二条才是致命的。我们真正想要的表示，必须让"相似的词落在相近的位置"——因为整个 Transformer 后面所有的运算，本质上都在比较向量之间的远近（第 5 章）。

所以需求很清楚：**稠密、低维、且几何距离携带语义**。

1.2 最小必要定义

本书对"向量"只采用最朴素的定义，不引入向量空间的公理体系。

向量：d 个实数按固定顺序排成一列，记作 $\mathbf{x} \in \mathbb{R}^d$ 。

它只支持两种运算：- 加法： $(\mathbf{a} + \mathbf{b})_i = a_i + b_i$ - 数乘： $(\lambda \mathbf{a})_i = \lambda \cdot a_i$

这就够了。整本书不会用到比这更抽象的东西。

嵌入矩阵： $\mathbf{E} \in \mathbb{R}^{V \times d}$ ，V 是词表大小，d 是嵌入维度。第 i 行就是第 i 个 token 的向量表示。

这些数不是人设定的，是训练出来的。E 是模型参数的一部分，和其他权重一样通过梯度下降更新。

行约定（本书全程遵守，请现在就固定下来）：

一行 = 一个 token。一个长度为 n 的句子对应 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 。

这个约定看似琐碎，但它决定了后面所有矩阵乘法的写法。教材里常见的"列向量"约定会让公式全部转置，与真实代码对不上。

一个值得记住的等价关系

查表取第 i 行，等价于用 one-hot 向量左乘嵌入矩阵：

$$\text{onehot}(i) @ \mathbf{E} == \mathbf{E}[i]$$

也就是说，"查表"其实是矩阵乘法的一个特例。实际实现当然直接查表（省掉 50257 次乘零加法），但认识到这一点很重要：它意味着嵌入层和其他线性层没有本质区别。这条线索会在第 2 章被正式展开。

1.3 直觉一：坐标没有意义，方向才有

初学者最常见的误解是：既然向量有 768 维，那第 37 维大概代表某种属性，比如"复数性"或"褒贬"。

这个想法是错的，而且错得很彻底。

看一个手工造的二维例子：

词	向量
cat	(0.9, 0.4)
dog	(0.8, 0.5)
democracy	(-0.3, 0.9)

计算余弦相似度（两个向量夹角的余弦，第 5 章会正式定义，这里先当作"像不像"的度量）：

- $\cos(\text{cat}, \text{dog}) \approx 0.99$ —— 几乎同向
- $\cos(\text{cat}, \text{democracy}) \approx 0.10$ —— 近乎垂直

符合直觉。现在把三个向量**同时**旋转 90° （把 (x, y) 换成 $(-y, x)$ ）：

词	旋转后
cat	$(-0.4, 0.9)$
dog	$(-0.5, 0.8)$
democracy	$(-0.9, -0.3)$

每一个坐标都变了，"第一维"的含义面目全非。但两个余弦值**一个都没变**，仍然是 0.99 和 0.10。

这说明了什么？**坐标轴的选择是任意的**。模型学到的不是"每一维代表什么"，而是"词与词之间的相对几何关系"。有意义的量是内积、夹角、差向量；单个坐标的数值本身几乎不可解释。

（顺带一提：保持内积不变的矩阵叫正交矩阵，旋转就是其中一种。第 10 章讲位置编码时会正面用到这个性质。）

差向量携带关系

同样的道理让下面这个经典现象成为可能：

$$\text{vec}(\text{king}) - \text{vec}(\text{man}) + \text{vec}(\text{woman}) \approx \text{vec}(\text{queen})$$

"性别"这个概念不住在某一个坐标里，而是住在**一个方向上**——从 man 指向 woman 的那个差向量。词嵌入时代对此有大量实证；Transformer 内部的情况更复杂，但这个直觉是有效的起点。

1.4 直觉二：方向管语义，长度管强度

一个向量可以拆成两部分信息：指向哪里（方向），有多长（范数 $\|x\|$ ）。

- **方向**大致对应"是什么意思"。
- **长度**大致对应"这个意思有多强、有多确定"。

这个分工不是修辞。到第 5 章会看到，注意力打分用的是**未归一化的内积** $q \cdot k = \|q\| \|k\| \cos \theta$ ——长度会直接放大或压低一个 token 被关注的程度。一个 token 可以通过增大自己的范数，让自己"更容易被看见"。这是模型可以利用的真实机制。

所以要注意：余弦相似度扔掉了长度信息，而 Transformer 内部并不扔。两者不要混为一谈。

1.5 直觉三：d 是一笔预算

d 该取多少？

模型	词表 V	嵌入维度 d
GPT-2 small	50257	768
GPT-2 XL	50257	1600
Llama-3-8B	128256	4096

d 太小，不同概念被迫挤在一起、互相干扰（信息瓶颈）；d 太大，参数量和计算量随之膨胀，且容易过拟合。d 是一笔关于“能同时表达多少东西”的预算。

这里埋一个伏笔。你可能会想：768 维最多只能容纳 768 个互相独立的概念吧？

并非如此，而且差得非常远。在高维空间里，“近似正交”（夹角接近 90° ，但不必恰好 90° ）的方向数量随维度**指数增长**。768 维可以容纳的近似正交方向数以百万计。这个反直觉的事实，是模型能在有限维度里塞进海量概念的根本原因。第 8 章会专门处理它。

1.6 一个容易忽略的点：嵌入只是起点

必须澄清一个术语混淆。

刚查表得到的 X 是**静态**的：同一个 token 在任何句子里查到的向量完全相同。此时的 "bank" 无法区分河岸和银行。

但这只是第 0 层的值。经过每一层之后，每个位置上的向量都会被改写（准确说是被“加上”新东西，第 8 章）。到了深层，“bank”的向量已经融合了上下文，成为**上下文相关**的表示。

所以：**嵌入表提供起点，网络负责把它变成真正的含义**。

另一端还有一个对称的操作。模型最后要预测下一个词，需要把 d 维向量变回 V 维的分数：

$$\text{logits} = \mathbf{h} @ \mathbf{W}_u \quad \# (n, d) @ (d, V) \rightarrow (n, V)$$

很多模型让 $\mathbf{W}_u = \mathbf{E}^T$ ，即输入嵌入和输出嵌入共享参数（权重绑定 / weight tying）。从几何上看，这是在说：“用哪个向量表示这个词”和“用哪个向量检测这个词”应该是同一个方向。

1.7 动手：十行代码

```
import numpy as np
rng = np.random.default_rng(0)
V, d = 6, 4                                # 词表 6 个词，嵌入维度 4
E = rng.normal(size=(V, d))                # 嵌入矩阵：一行一个词
ids = [2, 0, 5]                             # 一句话的三个 token id
X = E[ids]                                  # 查表 → (3, 4)，一行一个 token
print(X.shape)                             # (3, 4)
print(np.allclose(np.eye(V)[ids] @ E, X))  # True: 查表 == one-hot 乘 E
cos = lambda a, b: a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(cos(E[0], E[1]), np.linalg.norm(E[0]))
```

建议做的两个小实验：

1. 造一个随机正交矩阵 Q (`np.linalg.qr(rng.normal(size=(d,d)))[0]`)，比较 E 和 $E @ Q$ 各自的两两余弦相似度矩阵。你会发现完全一致——这就是 1.3 节的结论在高维的版本。
2. 把 d 从 4 改成 2，再改成 64，观察随机向量之间的平均余弦相似度如何随 d 变化。它会迅速趋近 0。这是第 8 章的预告。

1.8 落回 Transformer

在真实代码里，本章内容对应的就是这一行：

```
self.wte = nn.Embedding(50257, 768)        # 就是矩阵 E
x = self.wte(input_ids)                    # (B, n) → (B, n, 768)
```

B 是 batch 大小，从这里开始，本书的每个张量都会带上它。

算一笔账：GPT-2 small 的嵌入矩阵有 $50257 \times 768 \approx 3860$ 万个参数。而整个模型是 1.24 亿参数——嵌入层一个人就占掉了近三分之一。这也是权重绑定值得做的原因：省掉输出端另一个 3860 万。

1.9 自测

1. 已知 `onehot(i) @ E == E[i]`，那实际实现为什么不用矩阵乘法？请从计算量和显存两个角度说明（提示：50257 次乘法里有多少次是乘 0）。
2. 如果把嵌入矩阵换成 $E @ Q$ (Q 是正交矩阵)，模型的行为会改变吗？如果保持输出完全不变，后续哪些权重需要同步调整？这个问题的答案对“某一维代表什么含义”这类说法意味着什么？
3. 某模型词表 32000、 $d = 4096$ 。（a）嵌入矩阵参数量是多少？（b）若采用权重绑定，能省下多少？（c）如果把 d 翻倍到 8192，嵌入层参数量如何变化？模型其他部分的参数量（约 $12d^2$ 每层，见第 12 章）又如何变化？哪一部分增长更快？

本章小结

- 一个 token = 一个 d 维实向量；一句话 = 一个 (n, d) 矩阵，一行一个 token。
- 嵌入矩阵 $E \in \mathbb{R}^{V \times d}$ 是学出来的参数，查表等价于 one-hot 乘 E 。
- 坐标是任意的，关系才是真的。有意义的量是内积、夹角、差向量。
- 方向承载语义，长度承载强度，注意力两者都用。
- 静态嵌入只是第 0 层；上下文含义由后续层构建。

本章刻意没讲

向量空间的八条公理、线性无关与张成的形式定义、基与维数的一般理论。前者在本书中永远用不到；后两者会在第 7 章讨论秩的时候，以“能不能被别人加权拼出来”的具体形式重新出现——那时它们有明确的用武之地（多头注意力的低秩瓶颈），学起来会容易得多。

下一章预告

本章末尾埋了一根线：查表就是矩阵乘法。

第 2 章会顺着这根线走到底，回答一个看起来平淡无奇、实际上是全书支点的问题： $C = AB$ 到底在做什么？你会看到同一个式子有三种完全不同的读法，而其中被大多数教材忽略的那一种，将在第 6 章直接把注意力机制变成一件显然的事。第 2 章是本书最厚的一章，值得慢读。

第2章 矩阵乘法的三种读法

本章要回答：C = AB 到底在做什么？

读完本章你能做到：看到任何一个矩阵乘法，立刻判断它属于"比较""混合"还是"变换"，并用对应的视角把它讲清楚。

重要提示：这是本书最厚的一章，也是唯一一章值得反复读的。后面十章每一章都在调用它。如果时间有限，宁可在别处省，不要在这里赶。

2.1 困惑：会算，不等于看得见

几乎每个人都会算矩阵乘法。给你两个矩阵，按行乘列，一个元素一个元素填进去，机械但可靠。

问题是，这种算法只告诉你**怎么算出来**，不告诉你**它在干什么**。

打开任何一个 Transformer 实现，你会看到密集的 @：

```
q = x @ W_q           # ①
scores = q @ k.transpose(-2, -1) # ②
out = attn @ v         # ③
h = act(x @ W_1) @ W_2 # ④
```

这四个 @ 是同一个运算，但它们做的事完全不同：①在**变换**，②在**比较**，③在**混合**，④先变换再变换。

如果你脑子里只有"行乘列"这一种理解，这四行看上去就是一模一样的四次矩阵乘法，你会失去对模型的全部直觉。

本章的目标就是给你三副眼镜，让你能在它们之间自由切换。

2.2 最小必要定义

$C = AB$ ，其中 $A \in \mathbb{R}^{a \times b}$ ， $B \in \mathbb{R}^{b \times c}$ ，结果 $C \in \mathbb{R}^{a \times c}$ ，且

$$C_{ij} = \sum_{k=1}^b A_{ik} B_{kj}$$

形状规则只有一条：**中间的维度必须相等，并在相乘后消失。**

$$(a \times b)(b \times c) \rightarrow (a \times c)$$

↑____↑
必须相等，然后消失

这一条会在第3章被单独训练成肌肉记忆。本章专注于"含义"。

贯穿全章的例子

下面这一对矩阵会以三种方式被解读三次。请把它抄下来，放在手边。

$$A = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 3 & 1 \end{bmatrix} \quad (2 \times 3) \quad B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad (3 \times 2)$$

$$C = AB = \begin{bmatrix} 3 & 1 \\ 1 & 4 \end{bmatrix} \quad (2 \times 2)$$

2.3 读法一：点积视角（比较）

$C_{ij} = A$ 的第 i 行 与 B 的第 j 列 的点积。

验证一下 C 的左上角： A 的第 0 行是 $(2, 0, 1)$ ， B 的第 0 列是 $(1, 0, 1)$ ，点积 $= 2 + 0 + 1 = 3$ 。✓

这是教科书的标准定义，也是大多数人唯一掌握的视角。它的**适用场景**很明确：

当你关心 C 的某一个具体元素时，用点积视角。

落回 Transformer

注意力打分就是这个视角的教科书式应用：

```
scores = Q @ K.transpose(-2, -1)    # (n, d) @ (d, n) → (n, n)
```

$scores[i][j]$ = Q 的第 i 行 · K 的第 j 行 = **第 i 个 token 的 query 与第 j 个 token 的 key 有多匹配**。

这里有两件事值得停下来想：

第一，为什么要转置？ 因为按照第1章的行约定， Q 和 K 都是"一行一个 token"。要让第 i 行和第 j 行做点积，必须把其中一个转成列。`K.transpose` 就是为此存在的，不是什么深奥操作。

第二，形状告诉你一切。 $(n, d) @ (d, n) \rightarrow (n, n)$ 。d 消失了，n 出现了两次。这说明结果的每个元素都是"一对 token 之间的关系"，与特征维度无关——d 已经被点积吃掉了。整个 Transformer 里只有这一个张量是 $n \times n$ 的，长上下文的显存问题全部源于它。

口诀：形状 (n, n) 意味着"token 对 token"。

2.4 读法二：线性组合视角（混合）

这一节是本书的支点。如果只能记住一页纸，记这一页。

(AB) 的第 i 行 = $\sum_k A_{ik} \cdot (B \text{ 的第 } k \text{ 行})$

换成人话：结果的每一行，是用 A 那一行的数字当系数，对 B 的各行做加权求和。

回到例子。C 的第 0 行怎么来的？

$$\begin{array}{lcl}
 \text{A 的第 0 行} & = & (2, 0, 1) \quad \leftarrow \text{这三个数是系数} \\
 \\
 \text{C 第 0 行} & = & 2 \times (1, 0) + 0 \times (0, 1) + 1 \times (1, 1) \\
 & = & (2, 0) \quad + \quad (0, 0) \quad + \quad (1, 1) \\
 & = & (3, 1) \quad \quad \quad \checkmark \\
 & \uparrow & \quad \quad \uparrow \quad \quad \uparrow \\
 & \text{B 第0行} & \quad \text{B 第1行} \quad \text{B 第2行}
 \end{array}$$

再看第 1 行：系数是 (0, 3, 1)，于是

$$\text{C 第 1 行} = 0 \times (1, 0) + 3 \times (0, 1) + 1 \times (1, 1) = (1, 4) \quad \checkmark$$

结果一致。但过程的含义完全不同：点积视角看到的是"逐个元素算出来"，线性组合视角看到的是"把 B 的几行按比例混起来"。

为什么这个视角被忽略，又为什么它最重要

传统教材从解线性方程组出发，那里关心的是单个未知数的值，所以点积视角够用。

而深度学习里，几乎所有矩阵乘法的语义都是"混合"——把一堆向量按权重揉在一起。用点积视角看这类操作，你只能看到一堆数；用线性组合视角看，你能立刻看出输出是由哪些输入、以什么比例合成的。

落回 Transformer：注意力的输出

$$\text{out} = \text{attn} @ V \quad \# (n, n) @ (n, d) \rightarrow (n, d)$$

其中 attn 每一行经过 softmax, 非负且和为 1。套用线性组合视角:

$$\text{out 的第 } i \text{ 行} = \sum_j \text{attn}[i][j] \cdot (\text{V 的第 } j \text{ 行})$$

也就是说, 第 i 个位置的输出, 是所有 token 的 value 向量的**加权平均**。

做一个具体的数: 设 3 个 token,

```
attn = [ 0.7  0.2  0.1 ]      V = [ 1  0 ]
      [ 0.1  0.1  0.8 ]          | 0  2 |
                                      [ 4  4 ]

out 第 0 行 = 0.7×(1,0) + 0.2×(0,2) + 0.1×(4,4) = (1.1, 0.8)
out 第 1 行 = 0.1×(1,0) + 0.1×(0,2) + 0.8×(4,4) = (3.3, 3.4)
```

注意 out 的两行都落在 V 那三行向量围成的三角形内部。这不是巧合: 系数非负且和为 1 的加权和叫**凸组合**, 结果必然落在原向量的凸包里。

由此可以直接推出一个关于注意力的重要事实:

注意力不创造新的方向, 它只在已有的 value 向量之间做插值。

第 6 章会把这件事展开讲透。但你现在就已经理解了它——因为它只是线性组合视角的一次直接应用。

落回 Transformer: FFN 的写回

```
h = act(x @ W_1)      # (n, 4d)
out = h @ W_2          # (n, 4d) @ (4d, d) → (n, d)
```

再用一次线性组合视角。但注意, 这次要换个方向读——**按列**:

$$(\text{AB}) \text{ 的第 } i \text{ 行} = \sum_k A_{\{ik\}} \cdot (\text{B 的第 } k \text{ 行})$$

这里 A 是激活值 h , B 是 W_2 。所以:

$$\text{输出} = \sum_k h_k \cdot (W_2 \text{ 的第 } k \text{ 行})$$

W_2 有 $4d$ 行, 每一行是一个 d 维向量。这意味着: W_2 的每一行是一个候选的"输出方向", 激活值 h_k 决定这个方向被写入多少。

这就是把 FFN 理解为"键值记忆"的数学基础：W_1 的行负责匹配（哪些模式被激活），W_2 的行负责输出（激活后写入什么）。第 11 章会正式处理。

2.5 读法三：批量变换视角（变换）

$X @ W$ = 对 X 的每一行，独立地施加同一个线性映射。

当左边的矩阵是"数据"（一行一个样本）、右边是"权重"时，这是最自然的读法。

$$q = x @ W_q \quad \# (n, d) @ (d, d_k) \rightarrow (n, d_k)$$

n 个 token，每个都被同一个 W_q 投影到 d_k 维。 W_q 对每个 token 一视同仁，不因位置而异。

一个关键推论：左乘和右乘做的事截然不同

现在把前两个视角并排放：

	形式	效果
右乘权重	$X @ W, (n,d)(d,d')$	每行独立变换，行与行之间毫无交流
左乘权重	$A @ X, (n,n)(n,d)$	每行是原来若干行的混合，跨行搬运信息

验证左边那句话： $X @ W$ 的第 i 行 = $\sum_k X_{ik} \cdot (W \text{ 的第 } k \text{ 行})$ 。只用到了 X 的第 i 行，其他行的数据一个也没进来。

这个观察直接给出了 Transformer 的整体分工：

左乘混 token，右乘混特征。

- 所有的 Linear 层 (W_q 、 W_k 、 W_v 、 W_o 、 W_{l1} 、 W_{l2}) 都是右乘——它们只在特征维度上工作，永远不让两个 token 说话。
- 整个模型里唯一的左乘是 $\text{attn} @ V$ 。

所以：Transformer 中 token 之间的全部通信，只发生在注意力那一步。别的地方一步也没有。

这句话解释了很多事：为什么没有注意力的模型无法处理上下文；为什么因果掩码放在注意力里就足以保证不看未来；为什么 FFN 可以对每个位置并行计算而毫无依赖。

一个视角切换，换来对整个架构的结构理解。这就是本章值得花时间的理由。

2.6 补充读法：外积求和（为第 7 章埋线）

还有第四种展开方式，本书只在讲秩的时候用到，这里先见一面：

$$AB = \sum_{k=1}^b (A \text{ 的第 } k \text{ 列}) \otimes (B \text{ 的第 } k \text{ 行})$$

其中 $\otimes$ 是外积：列向量乘行向量，得到一个矩阵。回到例子：

$$\begin{aligned} k=0: \begin{bmatrix} 2 \\ 0 \end{bmatrix} \otimes (1, 0) &= \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} \\ k=1: \begin{bmatrix} 0 \\ 3 \end{bmatrix} \otimes (0, 1) &= \begin{bmatrix} 0 & 0 \\ 0 & 3 \end{bmatrix} \\ k=2: \begin{bmatrix} 1 \\ 1 \end{bmatrix} \otimes (1, 1) &= \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \\ \text{和} &= \begin{bmatrix} 3 & 1 \\ 1 & 4 \end{bmatrix} \quad \checkmark \end{aligned}$$

每个外积都是一个"秩 1"矩阵（所有行互相成比例）。所以：

任何矩阵乘法，都是 b 个秩 1 矩阵的叠加。 b 就是中间那个消失的维度。

这一句现在看不出用处，但它正是第 7 章"为什么多头注意力受低秩瓶颈约束"的全部内容。记住它的位置就行。

2.7 怎么选视角

不必纠结哪种"正确"——它们是同一个式子。选择依据是**你此刻关心什么**：

你想知道	用哪个视角	典型场景
C 的某一个元素是多少	点积	注意力分数 $\text{scores}[i][j]$
C 的某一行由什么合成	线性组合	$\text{attn} @ V$ 、FFN 写回
整个操作对数据做了什么	批量变换	所有 Linear 层
C 的秩最多是多少	外积求和	多头瓶颈、LoRA（第 7 章）

熟练之后，你会发现自己是**先判断语义、再选视角**，而不是先算再理解。这就是本章想训练出的能力。

2.8 两个必须掌握的性质

转置： $(AB)^T = B^T A^T$

顺序会反过来。这条规则在推导反向传播和读注意力代码时会不断出现。

理解方式：转置就是把"一行一个 token"变成"一列一个 token"。既然 A 作用在行上，转置后它就必须移到右边去作用在列上。

结合律： $(AB)C = A(BC)$ ——数值相同，代价不同

结合律说结果一样，但计算量可能差几个数量级。

拿注意力举例。设 Q, K, V 都是 (n, d)：

写法	步骤	乘法次数
$(QK^T)V$	$(n,d)(d,n) \rightarrow (n,n)$ ，再 $(n,n)(n,d)$	$O(n^2d)$
$Q(K^TV)$	$(d,n)(n,d) \rightarrow (d,d)$ ，再 $(n,d)(d,d)$	$O(nd^2)$

代入真实数字：n = 4096, d = 64。

- 前者：约 $2 \times 4096^2 \times 64 \approx 21.5$ 亿次乘法
- 后者：约 $2 \times 4096 \times 64^2 \approx 0.34$ 亿次乘法

差 64 倍。而且前者要存一个 4096×4096 的中间矩阵，后者只要 64×64 。

那为什么真实的注意力不用后一种写法？

因为中间夹着 softmax：

$\text{softmax}(QK^T) V$ $\leftarrow$ softmax 是逐行的非线性操作

softmax 不是矩阵乘法，结合律对它无效。你无法把 V "移进" softmax 里面。这个非线性正是 $O(n^2)$ 复杂度的来源。

而这也正是"线性注意力"这一整个研究方向的出发点：想办法去掉或近似 softmax，让结合律重新可用，把 $O(n^2d)$ 换成 $O(nd^2)$ 。

一条结合律，串起了标准注意力的代价与一整类高效注意力的动机。

顺带： $AB \neq BA$

矩阵乘法不可交换。这不是数学上的怪癖——它意味着"先注意力后 FFN"和"先 FFN 后注意力"是两个不同的模型。层的顺序是架构设计的一部分，不能随意调换。

2.9 动手

```
import numpy as np
A = np.array([[2., 0., 1.],
              [0., 3., 1.]])          # (2, 3)
B = np.array([[1., 0.], [0., 1.], [1., 1.]]) # (3, 2)

print(A @ B)                        # [[3 1] [1 4]]
print(A[0] @ B[:, 0])               # 读法一: C[0,0] = 3.0
print(A[0, 0]*B[0] + A[0, 1]*B[1] + A[0, 2]*B[2]) # 读法二: C 第 0 行 = [3 1]
print(sum(np.outer(A[:, k], B[k]) for k in range(3))) # 读法四: 秩 1 之和
```

建议做的三个实验：

1. 验证凸组合。用 2.4 节的 `attn` 和 `V` 算出 `out`，然后把 `V` 的三行和 `out` 的两行画在平面上（`matplotlib.pyplot.scatter`）。确认 `out` 落在三角形内部。再把 `attn` 某一行改成 `(1.5, -0.3, -0.2)`（仍然和为 1 但有负数），看 `out` 跑到哪里去了。
2. 验证"右乘不混 token"。造 `X = rng.normal(size=(5, 8))` 和 `W = rng.normal(size=(8, 8))`。计算 `X @ W`，然后只把 `X[2]` 改掉重算，比较两次结果——除了第 2 行，其余行应当逐位完全相同。再对 `A @ X` 做同样的实验，观察差别。
3. 验证结合律的代价差。用 `time.perf_counter()` 对 `n=2048`、`d=64` 的随机矩阵分别计时 `(Q@K.T)@V` 和 `Q@(K.T@V)`，同时用 `np.allclose` 确认两者数值一致。

第 2 个实验是本章最重要的动手环节，请务必做。

2.10 落回 Transformer：给每个 @ 贴标签

现在回头看章首那段代码，逐行标注：

```
q = x @ W_q          # 【批量变换】每个 token 独立投影，不跨 token
k = x @ W_k          # 【批量变换】
v = x @ W_v          # 【批量变换】

scores = q @ k.transpose(-2, -1) # 【点积】(n,d)(d,n)→(n,n)，token 对 token 打分
attn = softmax(scores / d**0.5)  # 非线性，结合律在此断开

out = attn @ v        # 【线性组合】唯一的左乘：token 之间的信息在这里流动
out = out @ W_o       # 【批量变换】
h = act(out @ W_1) @ W_2 # 【批量变换 → 线性组合写回】
```

整段代码里，只有一行是左乘。这就是整个 Transformer 的通信全部。

2.11 自测

1. 设 A 的第 3 行是 (0.5, 0.5, 0, 0)，B 是任意 (4, d) 矩阵。用一句话说出 (AB) 的第 3 行是什么。若把系数改成 (2, -1, 0, 0) (和仍为 1，但有负数)，结果的几何位置和前一种有什么区别？
2. Q、K、V 形状都是 (n, d)， $n = 8192$ ， $d = 128$ 。(a) 分别估算 $(QK^T)V$ 与 $Q(K^TV)$ 的乘法次数，给出比值。(b) 分别估算两者需要存储的最大中间张量元素个数。(c) 用一句话说明为什么标准注意力不能采用第二种算法。
3. 证明： $X @ W$ 的第 i 行只依赖于 X 的第 i 行。(提示：写出 (XW) 第 i 行的线性组合展开式，检查里面出现了 X 的哪些行。)在此基础上说明：如果一个模型只有 Linear 层和逐元素非线性，它能否完成"把第 5 个 token 的信息传给第 2 个 token"这件事？
4. (思考题) 残差连接 $x + f(x)$ 可以写成 $x @ I + f(x)$ ，其中 I 是单位矩阵。用批量变换视角说明：单位矩阵作为一个线性映射，在做什么？为什么说残差连接提供了一条"信息高速公路"？

本章小结

- 同一个 $C = AB$ ，有三种主要读法：**点积**（比较单个元素）、**线性组合**（一行由哪些行合成）、**批量变换**（对每行施加同一映射）；外积求和是留给第 7 章的第四种。
- 最重要的是线性组合视角：**(AB) 第 i 行 = $\sum_k A_{ik} \times$ (B 第 k 行)**。
- **左乘混 token，右乘混特征**。Transformer 里唯一的左乘是 $\text{attn} @ V$ ，所以 token 之间的全部通信都发生在那一步。
- 结合律不改变结果但改变代价；softmax 的存在切断了结合律，这正是注意力 $O(n^2)$ 的根源。
- 矩阵乘法不可交换，所以层的顺序是有意义的架构选择。

本章刻意没讲

行列式、矩阵求逆、高斯消元、分块矩阵的一般理论。

理由很直接：**Transformer 的前向传播里没有解方程，也没有求逆**。这些工具服务的是"已知 A 和 b，求 x"这类问题，而神经网络从头到尾只做"已知 A 和 x，求 Ax"。方向相反，用不上。

（分块矩阵的思想会在第 7 章讲多头拆分时以最朴素的形式出现——把一个大矩阵按列切成 h 段——但不需要任何一般理论。）

下一章预告

你现在理解了矩阵乘法在做什么。但真实代码里的张量是**四维**的： (B, h, n, d_k) 。

第 3 章会训练一项看起来低级、实际决定阅读速度的技能：**在不运行代码的情况下，说出每一步的形状**。这一章会很短、很枯燥、纯练习，但它是从"看懂公式"到"看懂代码"之间的那道门。

练完第 3 章，第一部就结束了，你将具备读懂注意力所需的全部工具。第二部会一口气把它读完。

第 3 章 形状代数

本章要回答：怎么在不运行代码的情况下，说出每一步的输出形状？

读完本章你能做到：手写出 Transformer 一层里全部 17 个中间张量的形状；解释多头拆分为什么必须 transpose；识别三类最常见的形状 bug。

本章性质：这是全书最枯燥的一章，也是最短的一章之一。它不引入任何新思想，只训练一项技能。但这项技能是“看懂公式”和“看懂代码”之间的那道门——很多人卡在这里，且往往不知道自己卡在这里。

3.1 困惑：公式是二维的，代码是四维的

第 2 章的所有讨论都发生在二维世界：X 是 (n, d) ，W 是 (d, d') 。

打开真实代码，你看到的是这个：

```
q = q.view(B, n, self.h, self.d_k).transpose(1, 2)
scores = q @ k.transpose(-2, -1)
scores = scores.masked_fill(mask[:, :, :n, :n] == 0, float('-inf'))
```

`view`、`transpose(1, 2)`、`transpose(-2, -1)`、`mask[:, :, :n, :n]`——公式里一个都没有。

它们从哪来？答案是两个在数学推导中被省略、在工程实现中无法省略的维度：

- **B (batch)**：一次处理多个句子。
- **h (head)**：一次并行多个注意力头。

于是二维的 (n, d) 变成了四维的 (B, h, n, d_k) 。公式没变，但代码里每一步都要照顾这两个多出来的维度。

本章就是训练你在四维世界里不迷路。

3.2 最小必要定义

规则一：矩阵乘法只作用于最后两维

这是理解一切的关键。对于高维张量，`@` 的行为是：

把最后两维当作矩阵，前面所有维度都视作 batch 维，逐个独立相乘。

$$\begin{array}{ccc} (B, h, n, d) @ (B, h, d, m) & \rightarrow & (B, h, n, m) \\ \text{——batch——} & \text{——矩阵——} & \text{前面原样保留} \end{array}$$

也就是说，一次 @ 实际上做了 $B \times h$ 次独立的矩阵乘法。

推论（非常重要）：如果两个维度需要参与矩阵乘法，它们必须被摆到最后两位。这就是所有那些 transpose 存在的唯一理由。

规则二：中间维消失

和第 2 章一样： $(a, b) @ (b, c) \rightarrow (a, c)$ 。这条规则现在只应用于最后两维。

规则三：广播 (broadcasting)

形状不同的张量做逐元素运算时：

1. 从右向左对齐维度；
2. 缺失的维度补 1；
3. 长度为 1 的维度被复制对方的长度；
4. 其余情况报错。

```
scores (B, h, n, n)
mask   (1, 1, n, n)    ← 右对齐后，前两维都是 1
      ↓
相加 → (B, h, n, n)    ← mask 被复制到每个 batch、每个 head
```

因果掩码之所以能写成 `scores + mask`，全靠这条规则。一个 $(1, 1, n, n)$ 的掩码自动服务于所有样本和所有头。

3.3 核心技能：三步追踪法

拿到任何一行代码，按这三步走：

1. 写下每个输入的形状（不要跳过，写出来）。
2. 找到会消失的中间维（矩阵乘法）或会被复制的 1（广播）。
3. 写下输出形状。

举例: `attn @ v`, 其中 `attn` 是 (B, h, n, n) , `v` 是 (B, h, n, d_k) 。

第 1 步: (B, h, n, n) 和 (B, h, n, d_k)
 第 2 步: 最后两维是 (n, n) 和 (n, d_k) , 中间的 n 消失
 第 3 步: (B, h, n, d_k)

再举一例: `q @ k.transpose(-2, -1)`, `q` 和 `k` 都是 (B, h, n, d_k) 。

第 1 步: `k` 转置后是 (B, h, d_k, n)
 第 2 步: (n, d_k) 与 (d_k, n) 相乘, d_k 消失
 第 3 步: (B, h, n, n)

看到 (n, n) 就要警觉——这是整个模型里唯一随序列长度平方增长的张量。

3.4 四个高频形变操作

reshape / view: 重新分组, 不搬数据

`reshape` 只改变"怎么读这堆数", 元素在内存里的顺序不变。前提是元素总数不变。

多头拆分就是一次 reshape:

$(B, n, d) \rightarrow (B, n, h, d_k)$ 其中 $d = h \times d_k$

`d` 这一维被切成 `h` 段, 前 `d_k` 个数属于第 0 个头, 接下来 `d_k` 个属于第 1 个头, 以此类推。这不是约定俗成, 而是 reshape 的必然结果——最后一维变化最快。

transpose / permute: 交换维度, 改变语义

拆完之后必须再做一步:

`q = q.view(B, n, h, d_k).transpose(1, 2)` # $(B, n, h, d_k) \rightarrow (B, h, n, d_k)$

为什么必须交换? 回到规则一: 矩阵乘法只作用于最后两维。我们要做的是 $(n, d_k) @ (d_k, n)$, 所以 `n` 和 `d_k` 必须落在最后两位, `h` 必须挪到前面去当 batch 维。

h 挪到前面 = 让"每个头"变成一个独立的批次。

这一步做完, `h` 个头就彻底互不干扰地并行了。第 7 章讲多头时会回到这个结论的语义含义; 此处只需接受它的形状必要性。

合并回来：transpose 再 reshape

注意力算完之后要把头拼回去：

```
out = out.transpose(1, 2).reshape(B, n, d)  # (B,h,n,d_k) → (B,n,h,d_k) → (B,n,d)
```

顺序不能颠倒。 如果直接对 (B, h, n, d_k) 做 reshape 到 (B, n, d) ，得到的数会是错位的——不同头的数据会被塞进同一个 token 的位置。这类 bug 不会报错，只会让模型永远训不好。

(PyTorch 里 transpose 之后张量不再连续，所以常见写法是 `.transpose(1,2).contiguous().view(...)`，或者直接用 `.reshape()` 让它自动处理。这是实现细节，不是数学问题。)

squeeze / unsqueeze：增删长度为 1 的维度

主要用于给广播"对齐"。 `mask.unsqueeze(0).unsqueeze(0)` 把 (n, n) 变成 $(1, 1, n, n)$ 。

3.5 einsum：把形状写在脸上

上面这些 transpose 很容易写错，而且写完就看不懂了。 `einsum` 用显式下标绕过全部心智负担：

```
scores = np.einsum('bhqd,bhkd->bhqk', Q, K)  # 不需要任何 transpose
out     = np.einsum('bhqk,bhkd->bhqd', A, V)
```

读法：重复且不出现在输出中的下标被求和消去。

第一行里 `d` 在两个输入中都出现、在输出中消失，所以沿 `d` 求和——这就是点积。`q` 和 `k` 都保留，得到 (n, n) 的分数矩阵。

第二行里 `k` 被消去，`q` 和 `d` 保留。

这两行就是完整的注意力核心。不需要记住任何 transpose 的参数，下标本身说明了一切。

建议：初学阶段用 `einsum` 写一遍，确认自己理解了；正式代码里两种都要能读，因为主流实现用的是 transpose 写法。

3.6 三类最常见的形状 bug

Bug 1：静默广播

```
a = np.zeros((5, 1))
b = np.zeros(5)
(a + b).shape  # (5, 5) ← 不是 (5,)! 
```


你以为在做逐元素加法，实际上得到了一个外积形状的矩阵。**它不会报错**，只会让下游的一切悄悄变错。

预防：凡是逐元素运算，前后都 assert 一下形状。

Bug 2: reshape 与 transpose 顺序颠倒

见 3.4 节。数量对得上，语义全错。**也不会报错**。

Bug 3: softmax 的维度选错

```
attn = softmax(scores, dim=-1)    # 正确：每个 query 对所有 key 归一化
attn = softmax(scores, dim=-2)    # 错误：变成每个 key 对所有 query 归一化
```

两者形状完全相同，模型照常训练，效果就是上不去。

三个 bug 的共同点：形状检查全部通过，程序不报错。这就是为什么形状追踪必须成为习惯——编译器不会替你做这件事。

3.7 动手

```
import numpy as np
B, n, d, h = 2, 6, 12, 3
d_k = d // h
x = np.random.randn(B, n, d)
W = np.random.randn(d, d)

q = x @ W                                # (2, 6, 12) 批量变换
q = q.reshape(B, n, h, d_k)             # (2, 6, 3, 4) 拆头
q = q.transpose(0, 2, 1, 3)              # (2, 3, 6, 4) n, d_k 移到最后两维
s = q @ q.transpose(0, 1, 3, 2)          # (2, 3, 6, 6) ← 出现 (n, n)
print(q.shape, s.shape)
print(np.allclose(s, np.einsum('bhqd,bhkd->bhqk', q, q))) # True
```

建议做的三个实验：

1. **感受静默广播**。亲手运行 3.6 节 Bug 1 的两行，确认它真的返回 (5, 5) 且不报错。然后想想：如果这发生在你的 attention 里，你多久能发现？
2. **验证拼头顺序**。对 (B, h, n, d_k) 分别执行"先 transpose 再 reshape"和"直接 reshape"，比较两个结果。用 np.allclose 确认它们不相等，再打印出来看看数字是怎么错位的。
3. **算一笔显存账**。对 B=8、h=12、n=1024，注意力矩阵有多少个元素？以 float32（4 字节）计需要多少 MB？把 n 换成 4096 再算一次，比较两个结果。（答案在下一节。）

3.8 落回 Transformer：一层的完整形状表

配置取 GPT-2 small: $B=2$, $n=1024$, $d=768$, $h=12$, $d_k=64$ 。

#	操作	输出形状
0	输入 x	$(2, 1024, 768)$
1	LayerNorm	$(2, 1024, 768)$
2	$x @ W_q$	$(2, 1024, 768)$
3	reshape 拆头	$(2, 1024, 12, 64)$
4	$\text{transpose}(1, 2)$	$(2, 12, 1024, 64)$
5	k 、 v 同样处理	$(2, 12, 1024, 64)$
6	$q @ k^T$	$(2, 12, 1024, 1024)$
7	$\div \sqrt{64}$, $+ \text{mask}$ $(1,1,1024,1024)$	$(2, 12, 1024, 1024)$
8	$\text{softmax}(\text{dim}=-1)$	$(2, 12, 1024, 1024)$
9	$@ v$	$(2, 12, 1024, 64)$
10	$\text{transpose}(1, 2)$	$(2, 1024, 12, 64)$
11	reshape 拼头	$(2, 1024, 768)$
12	$@ W_o$	$(2, 1024, 768)$
13	残差相加	$(2, 1024, 768)$
14	LayerNorm	$(2, 1024, 768)$
15	$@ W_1$	$(2, 1024, 3072)$
16	GELU	$(2, 1024, 3072)$
17	$@ W_2$	$(2, 1024, 768)$
18	残差相加	$(2, 1024, 768)$

请注意两件事。

第一，第 0 步和第 18 步形状完全相同。这不是巧合，而是设计要求——只有输入输出同形，层才能无限堆叠。GPT-2 把这个 $(2, 1024, 768)$ 的张量原样传递了 12 次。第 8 章讲残差流时，这个观察会成为主角。

第二，第 6~8 步的 $(2, 12, 1024, 1024)$ 是唯一的异类。它有 $2 \times 12 \times 1024 \times 1024 \approx 2517$ 万个元素，float32 下约 100 MB——而这只是一层里的一个张量。

把 n 从 1024 换成 4096：元素数变成约 4.03 亿，约 1.6 GB。序列长了 4 倍，这个张量大了 16 倍。

长上下文的全部工程困难，都写在这一行的形状里。

(这也是 FlashAttention 这类工作的出发点：它们的核心贡献不是减少计算量，而是**避免把这个 (n, n) 张量完整地写进显存。**)

3.9 自测

前四题请**不要运行代码**，手写答案后再验证。

- 写出下列每一步的输出形状： (a) $(4, 128, 512) @ (512, 2048)$ (b) $(4, 8, 128, 64) @ (4, 8, 64, 128)$
(c) $(4, 8, 128, 128) @ (4, 8, 128, 64)$ (d) $(4, 8, 128, 128) + (1, 1, 128, 128)$ (e) $(4, 128, 1) + (128,)$ ← 小心
- 某模型 $d=4096$, $h=32$ 。(a) d_k 是多少？(b) 写出从 $(B, n, 4096)$ 到 $(B, 32, n, 128)$ 的两步操作及各自的中间形状。(c) 如果误把 `reshape` 写成 (B, n, d_k, h) ，形状检查会通过吗？模型会报错吗？后果是什么？
- `einsum('bnd,de->bne', x, W)` 在做什么？用第 2 章的三种读法之一给它命名，并写出对应的普通 `@` 表达式。
- 某人把 `softmax(scores, dim=-1)` 写成了 `dim=-2`。(a) 输出形状变了吗？(b) 从语义上说，这两种归一化分别在断言"什么加起来等于 1"？(c) 如果模型仍然能训练但效果明显偏低，你会怎么定位这个 bug？
- (综合) $B=1$, $n=8192$, $d=1024$, $h=16$ ，共 32 层。仅考虑注意力矩阵（第 6~8 步），一次前向传播中所有层加起来需要多少显存 (float32)？如果改用 float16 呢？这个数字对"能否在 24 GB 显卡上跑推理"意味着什么？

本章小结

- 矩阵乘法只作用于**最后两维**，前面的全是 batch 维。所有 transpose 的存在理由都是"把该相乘的两维挪到最后"。
- 广播规则：右对齐、补 1、复制。掩码 $(1,1,n,n)$ 靠它服务全部样本与全部头。
- 多头拆分是 `reshape + transpose(1,2)`；拼回是 `transpose(1,2) + reshape`。顺序颠倒不会报错，但结果全错。
- 三类静默 bug：意外广播、形变顺序错、softmax 维度错。它们的共同特征是**形状检查全部通过**。
- 一层的输入与输出形状必须相同，这是层可堆叠的前提。
- (n, n) 是模型里唯一随序列长度平方增长的张量，长上下文的困难全在这里。

本章刻意没讲

张量的内存布局（stride、contiguous）、自动微分中的形状传播、`vmap` 与函数式批处理、稀疏张量格式。

这些属于工程实现而非线性代数。知道 `.contiguous()` 的存在就够了；真正需要时，它们是半小时能查明白的东西，不值得占用一本极简教材的篇幅。

第一部结束

到这里，你已经具备读懂注意力所需的全部工具：

- 第 1 章给了你对象——向量与 (n, d) 矩阵。
- 第 2 章给了你运算的三种含义——比较、混合、变换。
- 第 3 章给了你在四维世界里不迷路的能力。

第二部会用四章把注意力彻底拆开。而你会发现进展比预期快得多——因为最难的部分已经在第 2 章讲完了。

第 4 章先处理一个问题：`x @ w_q` 把 768 维压到 64 维，这个"压缩"到底丢掉了什么，又为什么值得？

顺便，它还会给出一个看似简单却致命的论证：若干个线性变换叠在一起，仍然只是一个线性变换。这句话将决定整个模型的架构必须长成什么样。

第 4 章 线性变换与投影

本章要回答：把 768 维压到 64 维，丢掉的东西去哪了？为什么模型不崩？

读完本章你能做到：解释 W_q 在几何上做了什么；说清“降维不是损失而是提问”；给出“纯线性网络的深度毫无意义”的完整论证。

本章有一个隐藏的重量级结论，出现在 4.5 节。它只用第 2 章的结合律就能证明，却决定了整个 Transformer 的架构必须长成现在这样。

4.1 困惑：92% 的维度去哪了

GPT-2 里，每个 token 是 768 维向量。而每个注意力头的 query 只有 64 维：

$$q = x @ W_q \quad \# (n, 768) @ (768, 64) \rightarrow (n, 64)$$

768 个数变成 64 个数。92% 的维度不见了。

按朴素的信息论直觉，这应该是灾难性的：一个 768 维空间无法被无损地塞进 64 维，一定有东西丢了。而且模型不只丢一次——每一层的每一个头都在做这件事。

那为什么模型不仅没崩，反而工作得非常好？

这个问题的答案，会顺带解释多头注意力为什么要存在。

4.2 最小必要定义

线性变换：一个映射 f ，满足

- $f(a + b) = f(a) + f(b)$
- $f(\lambda a) = \lambda \cdot f(a)$

只有这两条。翻译成人话：先加再变换，等于先变换再加；缩放也一样。

这个定义看起来抽象，但它有一个极其实用的后果：

在固定了坐标之后，每一个线性变换都对应唯一一个矩阵；反过来也一样。

所以"线性层"和"矩阵"是同一件事的两个名字。PyTorch 里的 `nn.Linear` 就是一个矩阵，不多不少。

矩阵的行告诉你基向量去了哪

按本书的行约定（ x 是行向量，变换写作 $x @ W$ ），把标准基向量代进去：

$$\begin{aligned} e_0 &= (1, 0, 0, \dots) \rightarrow e_0 @ W = W \text{ 的第 0 行} \\ e_1 &= (0, 1, 0, \dots) \rightarrow e_1 @ W = W \text{ 的第 1 行} \end{aligned}$$

（这直接来自第 2 章的线性组合视角： $x @ W = \sum_k x_k \cdot (W \text{ 第 } k \text{ 行})$ ，而 e_0 的系数只有第 0 位是 1。）

于是有了读矩阵的第一直觉：

W 的第 i 行 = 第 i 个基向量变换后的位置。

一个二维例子：

$$W = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} \quad \begin{aligned} e_0 &= (1, 0) \rightarrow (2, 0) \\ e_1 &= (0, 1) \rightarrow (1, 3) \end{aligned}$$

单位正方形的四个角 $(0,0)$ 、 $(1,0)$ 、 $(0,1)$ 、 $(1,1)$ 被映到 $(0,0)$ 、 $(2,0)$ 、 $(1,3)$ 、 $(3,3)$ ——一个平行四边形。

由此可以看出线性变换**保持什么**：原点不动、直线仍是直线、平行仍然平行、等分点仍然等分。以及**不保持什么**：长度和角度一般都会变（除非 W 是正交矩阵，见第 10 章）。

（面积被放大的倍数由行列式给出，但本书用不到它，所以不展开。）

顺带澄清：加了偏置就不是线性变换了

$x @ W + b$ 严格来说是**仿射变换**，不是线性变换——因为 $f(0) = b \neq 0$ ，原点被平移了。

这个区分在数学上重要，在实践中影响不大。值得知道的是：**很多现代模型（如 LLaMA 系列）干脆去掉了大部分 `bias`**，效果几乎不变。原因之一是每层前面都有归一化操作，它本身就带可学习的平移参数，`bias` 的作用被大量重复了。

本书后续讨论一律忽略 `bias`，不影响任何结论。

4.3 投影：降维到底丢了什么

$W_q \in \mathbb{R}^{768 \times 64}$ 把 768 维映到 64 维。这类"目标维度低于源维度"的映射，我们统称**投影**。

确实有东西被丢掉了

输出只有 64 维，而输入有 768 维。必然存在大量输入方向，它们的差别在输出端看不见——具体地说，至少有 $768 - 64 = 704$ 个维度的信息被压掉了。

用几何语言：存在一个至少 704 维的子空间（叫做**核**或**零空间**），里面所有向量都被映成同一个点。落在这个子空间里的差异，输出端完全无法分辨。

这不是缺陷，是维度算术的必然。

但"丢掉"是错误的框架

关键的认识转换在这里：

W_q 不是在压缩信息，而是在选择关心什么。

想象一团三维点云。从正上方投影到地面，你看到的是平面分布；从侧面投影，你看到的是高度分布。**两次投影都丢掉了 1/3 的维度，但丢掉的不是同一份。**

哪一个投影"更好"？取决于你想问什么问题。想知道东西堆得多高，就从侧面看；想知道摊得多开，就从上面看。

所以：

一次投影 = 一个提问角度。 W_q 定义的是"我打算在哪些方面比较两个 token"。

第 5 章会看到，注意力分数实际上是 $x_i^T (W_q W_k^T) x_j$ ——两个 token 先各自被投影到某个 64 维子空间，然后在那个子空间里比较。换个 W_q ，就是换个比较标准。

这直接解释了多头

如果每次投影都是一个提问角度，那么**多个头就是多个同时进行的提问**：

- 一个头可能投影到与句法结构相关的子空间，比较"谁是谁的主语"；
- 另一个头可能投影到与指代相关的子空间，比较"这个代词指向谁"；
- 还有的头处理位置邻近、词汇重复等等。

（这只是粗略图景。可解释性研究确实找到了功能相对明确的头——比如著名的"归纳头"——但绝大多数头的行为是混杂的，不要指望一一对应。）

重点在结构上：如果只有一个头，模型就只能用一个提问角度看全部内容。多头让它同时从 h 个角度看。第 7 章会用秩的语言把这件事讲得更精确。

4.4 那为什么不崩？高维空间的慷慨

回到 4.1 的困惑。既然确实丢掉了 704 个维度，为什么模型没事？

三个原因叠加：

第一，信息没有被丢弃，只是没被这个头使用。 残差流始终保持 768 维（第 3 章的形状表：每层的输入输出都是 768）。 W_q 的投影只影响"这个头怎么打分"，原始的 768 维向量一路完好地传下去。**投影是抽取，不是替换。**

第二， h 个头合起来覆盖回了全维度。 12 个头各取 64 维， $12 \times 64 = 768$ 。它们的投影方向不同，合起来仍有机会覆盖整个空间。

第三，也是最反直觉的一条：随机投影惊人地保距。

有一个叫 Johnson-Lindenstrauss 的结果，粗略地说：

把 N 个点随机投影到大约 $O(\log N / \epsilon^2)$ 维，它们两两之间的距离能以 $(1 \pm \epsilon)$ 的精度保持下来。**这个维度只和点的个数有关，与原空间维度无关。**

注意"与原空间维度无关"这句。这意味着从 768 维降到 64 维，和从 76800 维降到 64 维，损失是差不多的。**高维空间里的点本来就"很稀疏"，压缩它们比直觉中便宜得多。**

这条结果还给出了另一个视角来理解第 1 章埋下的伏笔：高维空间能容纳的近似正交方向数以指数增长，而正是这种"拥挤但互不干扰"的结构，让低维投影仍能保留大部分可分辨性。

建议在 4.6 的实验 2 里亲手验证一次——亲眼看到 768 维降到 64 维后余弦相似度几乎不变，比读十遍定理有用。

4.5 一个致命的论证：线性叠线性，还是线性

现在来到本章最重要的一节。它的全部工具是第 2 章的结合律。

假设我们搭一个"深度"网络，一层接一层，每层都是线性变换：

$$y = ((x @ W_1) @ W_2) @ W_3$$

用结合律重组：

$$y = x @ (W_1 W_2 W_3) = x @ W'$$

W' 只是一个普通矩阵。于是：

三层线性网络，等价于一层线性网络。

一百层也一样。没有非线性时，深度的贡献严格为零。

这不是近似、不是"效果差不多"，而是**数学上完全等价**——存在一个单矩阵，对任意输入都给出逐位相同的输出。

更糟的是，还可能不如单层

如果中间做了降维，情况更差。设 $W_1 \in \mathbb{R}^{768 \times 64}$ ， $W_2 \in \mathbb{R}^{64 \times 768}$ ，那么

$$\text{rank}(W_1 W_2) \leq \min(\text{rank } W_1, \text{rank } W_2) \leq 64$$

等效矩阵 W' 虽然是 768×768 ，但秩最多 64。它的表达力**严格弱于**一个自由的 768×768 矩阵。

（这条秩不等式来自第 2 章的外积求和视角： $W_1 W_2$ 是 64 个秩 1 矩阵之和。第 7 章会正式展开，那时它会变成理解多头瓶颈和 LoRA 的关键。）

推论：非线性是架构的必需品，不是装饰

既然纯线性的深度毫无意义，那么模型的全部表达力必然来自非线性。

数一数 Transformer 一层里的非线性操作，只有三处：

位置	操作	性质
注意力打分之后	softmax	非线性，且 依赖于所有 token （因为要归一化）
FFN 中间	GELU / SwiGLU	逐元素非线性
每个子层之前	LayerNorm / RMSNorm	除以自身的范数，也是非线性

其余的一切——所有 W_q 、 W_k 、 W_v 、 W_o 、 W_1 、 W_2 ，即模型 99% 以上的参数——做的都是线性的事。

这个对比值得停下来想一想：绝大部分参数负责"选择关心什么"（投影与混合），而全部的表达能力来自那几个数量上微不足道的非线性点。

顺带一提，softmax 那一处非线性还额外承担了两件事：第 2 章讲过它切断了结合律、造成 $O(n^2)$ 复杂度；第 6 章会讲它保证了权重非负且和为 1，从而让注意力输出成为凸组合。一个操作，三重角色。

第 11 章会正面处理"非线性到底带来了什么表达力"。这里只需要牢牢记住它**必须存在**。

4.6 动手

```
import numpy as np
rng = np.random.default_rng(0)
d, d_k = 768, 64
x = rng.normal(size=(4, d))
Wq = rng.normal(size=(d, d_k)) / np.sqrt(d)

q = x @ Wq                                     # (4, 64) 一次投影
W1, W2 = rng.normal(size=(d, 64)), rng.normal(size=(64, d))
print(np.allclose((x @ W1) @ W2, x @ (W1 @ W2))) # True: 两层线性 == 一层
print(np.linalg.matrix_rank(W1 @ W2))           # 64, 不是 768
```

建议做的三个实验：

1. **看见"等价于一层"**。上面第二行输出 True 之后，把层数加到 10 层，确认仍然 True。然后想清楚：如果你训练一个 10 层纯线性网络，梯度下降在优化什么？
2. **验证随机投影保距（本章最值得做的实验）**。造 50 个 768 维随机向量，计算两两余弦相似度矩阵。再用一个随机的 768×64 矩阵把它们投影到 64 维，重算余弦相似度矩阵。比较两个矩阵的差（`np.abs(A - B).max()`）。然后把目标维度依次换成 16、64、256，观察最大误差如何变化。你会看到误差下降得比"维度比例"暗示的快得多。
3. **看见核**。对 `Wq` (768×64) 计算 `d = np.linalg.matrix_rank(Wq)`，确认它是 704。然后用 `scipy.linalg.null_space(Wq.T)` 取一个零空间方向 `v`，验证 `v @ Wq` 确实接近全零——这就是"被压掉的方向"的具体样子。

4.7 落回 Transformer

```
self.W_q = nn.Linear(768, 768, bias=False) # 一个 768×768 矩阵
q = self.W_q(x)                             # (B, n, 768)
q = q.view(B, n, 12, 64).transpose(1, 2)    # (B, 12, n, 64) ← 12 个头
```

注意实现上的一个细节：代码里只有一个 768×768 的矩阵，不是 12 个 768×64 的矩阵。

这两种写法在数学上完全等价——把 12 个 $(768, 64)$ 的投影矩阵横向拼起来，就是一个 $(768, 768)$ 的矩阵。第 3 章的 `view + transpose` 干的就是把结果重新切成 12 份。

合并成一个大矩阵纯粹是为了效率（一次大矩阵乘法远快于 12 次小的）。理解模型时，请始终把它想成 12 个独立的 $768 \rightarrow 64$ 投影。

4.8 自测

1. $W \in \mathbb{R}^{768 \times 64}$ 。(a) 它的零空间维度至少是多少？(b) "至少 704 个方向的信息被压掉"这句话，为什么不意味着模型丢失了 704 维的信息？（提示：想想残差流。）
2. 证明 $(x @ W_1) @ W_2 = x @ (W_1 W_2)$ ，并回答：(a) 一个 100 层的纯线性网络，表达力等价于几层？(b) 若 $W_1 \in \mathbb{R}^{768 \times 64}$ 、 $W_2 \in \mathbb{R}^{64 \times 768}$ ，等效矩阵 W' 的秩上界是多少？(c) 据此说明：为什么"把一个大矩阵拆成两个小矩阵"既是 LoRA 的省参数手段，也是它的能力限制？
3. 列出 Transformer 一层中所有的非线性操作。如果把它们全部换成恒等映射，一个 96 层的模型会退化成什么？它还能完成语言建模任务吗？
4. `nn.Linear(768, 768)` 与 `nn.Linear(768, 768, bias=False)`：(a) 参数量差多少？(b) 前者是线性变换还是仿射变换？(c) 现代模型大量去掉 bias 而效果不变，你能给出一个可能的解释吗？
5. (思考题) 4.3 节说"一次投影 = 一个提问角度"。据此论证：为什么 12 个 64 维的头，可能比 1 个 768 维的头更有用？（这道题的完整答案在第 7 章，此处只需给出直觉。）

本章小结

- 线性变换 $\iff$ 矩阵。行约定下，**W 的第 i 行 = 第 i 个基向量的像**。
- 带 bias 的是仿射变换；实践中影响很小，现代模型常常省略。
- 投影会不可逆地压掉一个至少 $(d - d_k)$ 维的子空间，但**这是选择关心什么，不是丢失信息**——残差流始终保留完整的 d 维。
- 降维之所以代价可控，是因为高维空间中随机投影近似保距（Johnson–Lindenstrauss），所需维度只与点数有关，与原维度无关。
- **纯线性的深度贡献严格为零**：任意多层线性变换等价于单个矩阵；若中间降维，还会附带秩的损失。
- 因此模型的全部表达力来自三处非线性：softmax、激活函数、归一化。**其余 99% 的参数都在做线性的事**。

本章刻意没讲

线性变换的核与像的一般理论、秩-零化度定理的证明、行列式与体积、特征方向、一般的基变换与相似矩阵。

本章只需要"投影会压掉一个子空间"这一条事实性认识。它的精确版本（秩不等式）会在第 7 章以最需要它的形式出现——那时读者已经见过多头注意力，学起来有具体依托。

下一章预告

我们已经知道 W_q 和 W_k 把两个 token 投影到某个 64 维子空间。

第 5 章处理接下来的问题：投影完了之后，怎么衡量它们"像不像"？

这一章会给出全书三个正式推导中的第一个——**为什么要除以 $\sqrt{d_k}$** 。这个看似神秘的常数其实只是一次方差计算，五行就能推完。同时你会看到，注意力用的是未归一化的内积而非余弦相似度，这个选择让向量的**长度**也变成了一种可被模型利用的资源。

第 5 章 内积、范数与打分

本章要回答：两个向量"像不像"，该怎么量？为什么要除以 $\sqrt{d_k}$ ？

读完本章你能做到：独立推导 $\sqrt{d_k}$ 的来历；解释 query 范数和 key 范数各自控制什么；说清注意力为什么用内积而不用余弦相似度。

本章含全书三个正式推导中的第一个（5.5 节）。它只有五行，用到的全部概率知识是"独立随机变量之和的方差等于方差之和"。

5.1 困惑："像"有很多种意思

第 4 章结束时，两个 token 已经被投影到某个 64 维子空间，成了 q 和 k 。现在要给它们打一个分数，表示"第 i 个 token 应该多关注第 j 个"。

问题是，"像"至少有三种常见的量法：

内积	$q \cdot k$	← 注意力用的是这个
余弦相似度	$q \cdot k / (\ q\ \ k\)$	← 检索系统常用
欧氏距离	$\ q - k\ $	← 聚类常用

它们在数学上密切相关，但行为差别很大。选哪一个不是随手决定的，而是一个有后果的设计。

本章先把三者的关系理清，再说明 Transformer 为什么选了第一个，最后处理那个几乎所有人第一次看到都觉得神秘的 $\sqrt{d_k}$ 。

5.2 最小必要定义

内积： $a \cdot b = \sum_i a_i b_i$ （一个数，不是向量）

范数： $\|a\| = \sqrt{a \cdot a}$ （向量的长度）

夹角： $\cos \theta = (a \cdot b) / (\|a\| \|b\|)$

三者由一个恒等式串起来，这是本章最重要的一行：

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \cdot \|\mathbf{b}\| \cdot \cos \theta$$

内积同时编码了**两件事**：方向有多一致（ $\cos \theta$ ），以及两个向量有多长（ $\|\mathbf{a}\| \|\mathbf{b}\|$ ）。

请记住这个"两件事"，5.4 节整节都建立在它上面。

三种度量的关系

把范数展开：

$$\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2(\mathbf{a} \cdot \mathbf{b})$$

于是可以看出：如果所有向量都被归一化到单位长度（ $\|\mathbf{a}\| = \|\mathbf{b}\| = 1$ ），那么

$$\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2(\mathbf{a} \cdot \mathbf{b})$$

内积越大，距离越小，二者严格反向单调。也就是说：

在单位球面上，内积、余弦相似度、欧氏距离三者互相等价——用哪个都一样。

差别只在离开单位球面之后。而注意力恰恰不做归一化。

5.3 注意力打分的形状

```
scores = Q @ K.transpose(-2, -1)    # (n, d_k) @ (d_k, n) → (n, n)
```

`scores[i][j] = q_i · k_j`，这是第 2 章的点积视角。

把第 4 章的投影代进去，可以看到一个更完整的图景：

$$\text{scores}[i][j] = (\mathbf{x}_i \mathbf{W}_q) \cdot (\mathbf{x}_j \mathbf{W}_k) = \mathbf{x}_i (\mathbf{W}_q \mathbf{W}_k^T) \mathbf{x}_j^T$$

中间那个 $\mathbf{W}_q \mathbf{W}_k^T \in \mathbb{R}^{d \times d}$ 才是这个头真正学到的东西——它定义了"在这个头看来，什么样的 $\mathbf{x}_i$ 应该关注什么样的 $\mathbf{x}_j$ "。

$\mathbf{Q}$ 和 $\mathbf{K}$ 从来不单独起作用，起作用的永远是它们的乘积。（第 7 章会分析这个矩阵的秩，那是多头注意力存在的数学理由。）

5.4 长度是一种资源

现在回到 5.2 的"两件事"。注意力用的是未归一化的内积，所以范数会直接进入分数。这不是疏忽，而是给了模型两个可以独立使用的控制旋钮。

旋钮一：key 的范数 = 显著性

如果把某个 k_j 的长度乘以 2，那么只有第 j 列的分数被放大 2 倍。

对那些原本分数为正的 query 来说，第 j 个 token 变得更容易被注意到。粗略地说：

key 范数大 = "我很重要，看我"。

（注意这里有个前提：若原本分数为负，放大反而会让它更被忽略。所以这个旋钮的效果是"放大已有倾向"，而非无条件提升。）

旋钮二：query 的范数 = 锐度

如果把 q_i 的长度乘以 2，那么第 i 行的全部分数同时被放大 2 倍。

行内的相对大小没变，但差距被拉大了。经过 softmax 之后，分布会变得更尖锐——原本 0.4/0.3/0.3 的分配可能变成 0.7/0.15/0.15。

query 范数大 = "我要专注地看一个地方"；范数小 = "我大致平均地看"。

这两个旋钮的存在，正是余弦相似度做不到的。归一化会把它们一起抹掉。

一个真实的后果：注意力沉降

在训练好的模型里，人们观察到一个奇怪现象：大量注意力权重堆在序列开头的某几个 token 上（常常是第一个 token 或标点），尽管它们语义上毫无意义。这被称为注意力沉降（attention sink）。

一种理解是：softmax 强制每行权重和为 1，模型没有"什么都不关注"这个选项。当某个头这一步不想搬运任何信息时，它需要一个垃圾桶来倾倒这份必须花掉的注意力预算。范数机制正是它构造垃圾桶的手段之一。

这个现象也解释了为什么近年一些模型引入了 QK-Norm（打分前对 q 、 k 做归一化）：把这两个旋钮锁死，换取训练稳定性。这是一个真实的取舍，不是纯粹的改进。

5.5 正式推导： $\sqrt{d_k}$ 从哪来

现象

先看出什么问题。假设 q 和 k 的各个分量是独立的、均值 0、方差 1 的随机数（训练初始化时大致如此）。那么内积

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i$$

是 d_k 个随机数之和。它有多大？

五行推导

期望：由独立性， $E[q_i k_i] = E[q_i] \cdot E[k_i] = 0 \times 0 = 0$ ，所以

$$E[q \cdot k] = 0$$

方差：各项独立，方差可加。单项的方差是

$$\text{Var}(q_i k_i) = E[q_i^2 k_i^2] - 0^2 = E[q_i^2] \cdot E[k_i^2] = 1 \times 1 = 1$$

求和：

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} 1 = d_k$$

标准差：

$$\text{std}(q \cdot k) = \sqrt{d_k}$$

推导结束。全部用到的假设只有两条：**分量独立、方差为 1**。

为什么这是个问题

$d_k = 64$ 时，标准差是 **8**。这意味着一行分数里，最大值和典型值之间常常差好几个 8。

而 softmax 对输入尺度极度敏感——它算的是 $\exp$ 。两个分数差 8，概率比就是 $e^8 \approx 2981$ 倍。差 16，就是 890 万倍。

结果：softmax 输出退化成近似 one-hot，一个位置吃掉几乎全部权重。

这为什么糟糕？ 两个层面：

语义上，注意力失去了"混合多个来源"的能力，退化成硬性选择一个。第 6 章会看到，注意力的价值恰恰在于加权平均。

优化上，更致命。softmax 的雅可比矩阵是

$$\partial p_i / \partial s_j = p_i (\delta_{ij} - p_j)$$

当 p 接近 one-hot 时，每个 p_i 要么接近 0 要么接近 1， $p_i(1 - p_i)$ 全都趋近于 0。**梯度几乎完全消失，这一层学不动了。**

解法

把分数除以标准差：

$$\text{scores} = Q @ K.T / \sqrt{d_k}$$

方差从 d_k 回到 1，softmax 工作在敏感区间内，梯度正常。

为什么是 $\sqrt{d_k}$ 而不是 d_k

这是最常见的疑问。答案是：**要抵消的是尺度，而尺度由标准差刻画，不是方差。**

方差的量纲是"分数的平方"，标准差才和分数同量纲。除以 d_k 会矫枉过正——方差变成 $1/d_k$ ，分数被压得过小，softmax 反而趋近均匀分布，同样丢失区分能力。

它其实就是温度

如果你熟悉采样时的 temperature 参数：

$$\text{softmax}(s / T)$$

T 大则分布平坦， T 小则尖锐。那么 $\sqrt{d_k}$ 就是一个**固定的、由维度决定的温度**。它的唯一作用是把分数校准到 softmax 的工作区间。

诚实说明：这个假设未必一直成立

推导用到"分量独立、方差为 1"。这在随机初始化时成立，在训练之后不一定成立。

训练过的模型里， q 和 k 的分量往往高度相关，范数也远非 1。所以 $\sqrt{d_k}$ 是一个基于初始化的启发式常数，不是全程精确的归一化。它在实践中足够好用，但确实不完美——这正是 QK-Norm 那类方法出现的原因：它们不再依赖任何分布假设，直接在每一步强制归一化。

5.6 动手

```
import numpy as np
rng = np.random.default_rng(0)
d_k, n = 64, 8
q, K = rng.normal(size=d_k), rng.normal(size=(n, d_k))
s = K @ q                                # n 个分数
print(s.std())                           #  $\approx 8 = \sqrt{64}$ 

sm = lambda z: np.exp(z - z.max()) / np.exp(z - z.max()).sum()
p_raw, p_scaled = sm(s), sm(s / np.sqrt(d_k))
ent = lambda p: -(p * np.log(p + 1e-12)).sum()
print(p_raw.max(), p_scaled.max())        #  $\approx 1.0$  vs  $\approx 0.3$ 
print(ent(p_raw), ent(p_scaled), np.log(n)) #  $\approx 0$  vs  $\approx 1.8$  (上限  $\ln 8 \approx 2.08$ )
```

建议做的三个实验：

1. 看标准差随维度增长。把 d_k 依次设成 4、16、64、256，每次重复 1000 组随机 q 、 k ，画出内积标准差与 $\sqrt{d_k}$ 的对比。两条线应当重合。
2. 看熵的坍塌。固定 $d_k = 64$ ，把缩放因子从 $\sqrt{d_k}$ 逐渐改成 d_k^0 （不缩放）、 $d_k^{0.25}$ 、 $d_k^{0.5}$ 、 $d_k^{0.75}$ 、 d_k^1 ，每次记录 softmax 输出的熵。你会看到熵从接近 0 一路升到接近 $\ln n$ ——两端都不好，中间那个 0.5 才是校准点。这个实验能同时解释“为什么要缩放”和“为什么不能缩过头”。
3. 验证长度的两个旋钮。固定一组 q 、 K 。（a）把 q 乘以 2，观察 softmax 输出如何变尖锐。（b）改回原样，只把 $K[3]$ 乘以 2，观察第 3 个位置的权重变化——再试一次，这次选一个原本分数为负的位置，看看结果是否相反。

5.7 落回 Transformer

```
scores = q @ k.transpose(-2, -1) / math.sqrt(self.d_k)    # (B, h, n, n)
```

三件事值得注意：

第一，缩放用的是 d_k 而不是 d 。每个头独立打分，参与点积的是 64 维不是 768 维。写成 `math.sqrt(d_model)` 是一个真实存在的 bug，且不会报错。

第二，这一行是 $O(n^2)$ 张量诞生的地方。第 3 章算过： $n = 4096$ 时它约 1.6 GB。

第三，缩放必须在 softmax 之前、掩码之后（或之前）都保持一致。先 softmax 再缩放是完全不同的运算，没有意义。

5.8 自测

1. $d_k = 128$ 且分量满足推导假设。(a) 内积的标准差是多少？(b) 若某行里最大分数比次大分数高 2 个标准差，不缩放时两者的 softmax 概率之比约为多少？缩放后呢？
2. 完整重推 $\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$ 。明确指出在哪一步用到了"独立"，哪一步用到了"方差为 1"。如果 $\mathbf{q}$ 的分量方差是 σ^2 而不是 1，标准差应该是多少？缩放因子该怎么改？
3. 有人提议除以 d_k 而不是 $\sqrt{d_k}$ 。(a) 此时分数的方差是多少？(b) softmax 的输出会趋向什么？(c) 为什么这同样有害？
4. 写出 softmax 的雅可比 $\partial p_i / \partial s_j = p_i(\delta_{ij} - p_j)$ 。当 $\mathbf{p}$ 接近 one-hot 时它趋近什么？据此解释"分数太大导致学不动"这句话的确切含义。
5. (a) 把 q_i 的范数翻倍，第 i 行的注意力分布会怎样变？(b) 把 k_j 的范数翻倍呢？(c) 用这两条解释：为什么 QK-Norm 能提高训练稳定性，代价又是什么？
6. (思考题) 如果注意力改用余弦相似度打分，模型会失去哪些能力？"注意力沉降"现象还会以同样的形式出现吗？

本章小结

- $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$ ：内积同时编码方向一致性和长度。
- 在单位球面上，内积、余弦、欧氏距离三者等价；差别只出现在不归一化的时候。
- 注意力用未归一化内积，于是范数成为可用资源：**key 范数控制显著性，query 范数控制锐度**。余弦相似度会把这两个旋钮一起抹掉。
- 真正被学习的对象是 $\mathbf{W}_q \mathbf{W}_k^s$ ，Q 和 K 从不单独起作用。
- $\text{std}(\mathbf{q} \cdot \mathbf{k}) = \sqrt{d_k}$ ，推导只需"独立"和"方差为 1"两个假设。 $d_k = 64$ 时标准差为 8，足以让 softmax 饱和、梯度消失。
- 除以 $\sqrt{d_k}$ 把方差校准回 1。除以 d_k 会矫枉过正。这个常数本质上是一个固定温度。
- 该推导基于初始化时的分布假设，训练后未必成立——这是 QK-Norm 等方法的动机。

本章刻意没讲

内积空间的公理、柯西–施瓦茨不等式的证明、一般的范数族 (L_1 、 L_∞ 、Frobenius)、格拉姆矩阵与正定性、softmax 的数值稳定实现细节（减最大值那一步）。

本章只需要一个内积、一个范数、一条恒等式。其余在需要时再补，成本很低。

下一章预告

打分做完了，softmax 也做完了。剩下最后一步：

```
out = attn @ v
```

第 6 章会把这一行彻底讲透。而你会发现自己**其实已经懂了**——因为它只是第 2 章"线性组合视角"的一次直接应用。

那一章会很短。这是刻意的：一个概念如果在正确的地方铺垫过，兑现时应该是轻松的。届时你会看到注意力输出为什么必然落在 value 向量的凸包内、因果掩码为什么写成加 $-\infty$ 而不是乘 0，以及一句可以概括整个机制的话——**注意力不创造信息，它只搬运和混合**。

第 6 章 加权平均与凸组合

本章要回答： $\text{attn} @ v$ 这一行到底在做什么？

读完本章你能做到：说清注意力输出为什么必然落在 value 向量的凸包内；解释因果掩码为什么写成 $-\infty$ ；用一句话概括整个注意力机制。

本章很短。这是设计好的——它的全部数学工具在第 2 章就讲完了，本章只负责兑现。如果你读起来觉得“这不是显然的吗”，那说明第 2 章起效了。

6.1 你其实已经会了

先把第 2 章的那一行搬过来：

(AB) 的第 i 行 $= \sum_k A_{ik} \cdot (\text{B 的第 } k \text{ 行})$

现在把 A 换成注意力权重 attn ，把 B 换成 v ：

out 的第 i 行 $= \sum_j \text{attn}[i][j] \cdot (\text{V 的第 } j \text{ 行})$

完了。这就是注意力输出的全部内容。

本章剩下的篇幅，只是在追问一件事： attn 的系数有一个特殊性质，它带来了什么后果？

6.2 最小必要定义

softmax 保证了每一行权重非负，且和为 1。带这两个性质的加权和有专门的名字：

凸组合： 给定向量 $v_1, \dots, v_n$ 和系数 $a_1, \dots, a_n$ ，若

- $a_j \geq 0$ (非负)
- $\sum_j a_j = 1$ (和为一)

则 $\sum_j a_j v_j$ 称为它们的一个凸组合。

所有可能的凸组合构成的集合，叫这组向量的**凸包**——直观地说，就是把这些点用橡皮筋套住围出来的那块区域。

两个点的凸包是连接它们的线段，三个点的凸包是三角形， n 个点的凸包是一个多面体。

关键性质只有一条：

凸组合的结果，永远落在凸包内部（或边界上），绝不会跑到外面去。

6.3 注意力输出是凸组合

用第 2 章的例子。三个 token，value 是二维的：

```
attn = [ 0.7  0.2  0.1 ]      V = [ 1  0 ] ← v1
      [ 0.1  0.1  0.8 ]      [ 0  2 ] ← v2
                                [ 4  4 ] ← v3

out 第 0 行 = 0.7 · (1,0) + 0.2 · (0,2) + 0.1 · (4,4) = (1.1, 0.8)
out 第 1 行 = 0.1 · (1,0) + 0.1 · (0,2) + 0.8 · (4,4) = (3.3, 3.4)
```

两行系数都非负、都和为 1，所以两个输出都落在 v_1 、 v_2 、 v_3 围成的三角形内。

- 第 0 行的权重集中在 v_1 ，输出就靠近 v_1 。
- 第 1 行的权重集中在 v_3 ，输出就靠近 v_3 。

权重决定输出落在三角形的哪个位置。 这就是注意力做的全部事情：在已有的 value 之间选一个混合位置。

6.4 三个推论

推论一：注意力不创造新方向

输出必然落在 V 各行向量张成的范围内。它不可能指向一个所有 value 都没有的方向。

注意力是插值，不是生成。

这个限制比听上去严重：如果这一层的 value 里根本不包含某种信息，注意力再怎么加权也变不出来。信息必须先存在于某个位置，注意力才能把它搬过来。

推论二：输出的秩受限

$\text{out} = \text{attn} @ V$ ，而 out 的每一行都是 V 各行的线性组合。所以 out 的所有行都住在 V 的行空间里：

$$\text{rank}(\text{out}) \leq \text{rank}(V) \leq \min(n, d_v)$$

$d_v = 64$ 时，无论 n 多大，单个头的输出最多只有 64 维的变化空间。这是第 7 章的正题，此处先记下这个数字从哪来。

推论三：范数不会爆炸

用三角不等式加上系数和为 1：

$$\|\text{out}_i\| = \|\sum_j a_j v_j\| \leq \sum_j a_j \|v_j\| \leq (\sum_j a_j) \cdot \max \|v_j\| = \max \|v_j\|$$

注意力输出的长度，不会超过最长的那个 value。

这是一个免费的稳定性保证。深度网络里，"某一步会不会把数值放大"是核心关切——而注意力这一步天然安全，不需要任何额外约束。

对比一下：如果系数允许为负、或者和不为 1，这个保证立刻消失。

6.5 softmax 的三重角色

到这里可以把关于 softmax 的三条线索合并了。它在注意力里同时承担三件事：

角色	后果	出处
切断结合律	$(QK^T)V$ 无法重组为 $Q(K^TV)$ ，造成 $O(n^2)$ 复杂度	第 2 章
提供非线性	使深度有意义（否则整个模型塌成一个矩阵）	第 4 章
保证非负且和为一	使输出成为凸组合，带来推论一至三	本章

一个操作，三重身份。它是 **Transformer** 里最不可替换的组件——每一次尝试去掉它（线性注意力、sigmoid 注意力等），都要同时处理这三件事的后果。

如果允许负系数会怎样

把第一行权重改成 (1.5, -0.3, -0.2)，和仍然是 1，但有负数。这叫**仿射组合**，不是凸组合。算一下：

$$1.5 \cdot (1,0) - 0.3 \cdot (0,2) - 0.2 \cdot (4,4) = (0.7, -1.4)$$

三个 value 的纵坐标分别是 0、2、4，**全都非负**，而输出的纵坐标是 -1.4。它跑到三角形外面去了。

推论一、三同时失效：输出可以指向 value 里没有的方向，长度也不再有上界。

这不是说负权重一定是错的——确实有研究在探索这个方向。但它意味着你必须自己想办法处理稳定性，而 softmax 是免费给你的。

6.6 因果掩码：为什么是加 $-\infty$

自回归模型不能看未来，所以第 i 个 token 只允许关注 $j \leq i$ 的位置。实现方式是在 softmax **之前** 给分数加上一个矩阵：

$$\text{mask} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix} \quad \text{scores} = \text{scores} + \text{mask}$$

因为 $\exp(-\infty) = 0$ ，这些位置的权重精确变成 0。

为什么不能 softmax 之后再置零？

因为那样会破坏“每行和为 1”。

假设某行 softmax 后是 (0.5, 0.3, 0.2)，把后两个位置置零得到 (0.5, 0, 0)——**和只有 0.5 了**。此时 $\text{out} = 0.5 \cdot \mathbf{v}_1$ ，长度比 $\mathbf{v}_1$ 短了一半。这不是凸组合，推论三的保证没了，而且这个缩水因子会随位置变化，前面的 token 被系统性地压小。

结论：屏蔽必须发生在 log 空间（softmax 之前），概率空间的归一化才是干净的。

一般规律：**先屏蔽，后归一化**。顺序颠倒，归一化就白做了。

两个实现细节

第一，实践中常用 $-1e9$ 而不是 $-\text{inf}$ 。在 float16 下， $-\text{inf}$ 参与运算容易产生 NaN。一个足够大的负数效果相同且更安全。

第二，全被屏蔽的行会产生 NaN。如果某一行所有位置都是 $-\infty$ ，softmax 的分母是 0，结果是 0/0。这在处理 padding 时是真实的坑——一个全是 padding 的样本会让整个 batch 变成 NaN。

第三，掩码的形状是 $(1, 1, n, n)$ 。靠第 3 章的广播规则服务于所有 batch 和所有头。掩码本身与内容无关，只与位置有关，所以不需要为每个样本单独准备。

6.7 一句话概括整个注意力

现在可以把第 2 章到第 6 章串成一句话了：

注意力不创造信息，它只搬运和混合。

- **打分**（第 5 章）决定从哪里搬。
- **softmax** 把分数变成一组非负、和为一的搬运比例。
- **加权平均**（本章）执行搬运，结果落在已有 value 的凸包内。

那么信息从哪里来？

回到第 2 章那句话：**左乘混 token，右乘混特征**。注意力是模型里唯一的左乘，它负责跨位置通信；而真正对信息做**变换**的，是 FFN 那些右乘的矩阵。

注意力负责通信，FFN 负责计算。

这个分工是理解 Transformer 的最高层图景。第 11 章会从 FFN 那一侧把它讲完。

6.8 动手

```
import numpy as np
attn = np.array([[0.7, 0.2, 0.1], [0.1, 0.1, 0.8]])
V    = np.array([[1., 0.], [0., 2.], [4., 4.]])
out = attn @ V
print(out)                                # [[1.1 0.8] [3.3 3.4]]
print(attn.sum(axis=1))                   # [1. 1.] ← 凸组合的条件
print(np.linalg.norm(out, axis=1).max(),  # 前者 ≤ 后者：推论三
      np.linalg.norm(V, axis=1).max())
bad = np.array([[1.5, -0.3, -0.2]]) @ V  # 和仍为 1，但有负数
print(bad)                                # [[0.7 -1.4]] ← 跑出三角形了
```

建议做的三个实验：

1. **画出凸包（本章必做）**。用 `matplotlib` 把 V 的三行画成三角形，把 `out` 的两行画成点。确认点在内部。然后手动构造几组不同的权重（比如 $(1,0,0)$ 、 $(1/3,1/3,1/3)$ 、 $(0,0.5,0.5)$ ），看输出分别落在哪——顶点、重心、边的中点。**这个实验做完，凸组合就再也忘不掉了。**
2. **对比两种掩码方式**。给一行分数 $(2.0, 1.0, 0.5)$ ，屏蔽后两个位置。（a）加 $-1e9$ 后做 softmax，检查结果的和。（b）先 softmax 再把后两位置零，检查结果的和。比较两次的 `out`，并计算它们的范数差多少。
3. **制造一次 NaN**。把一整行分数全部设成 $-1e9$ ，然后 softmax。观察输出。再改成全 $-\text{inf}$ ，观察区别。（这是真实生产事故的最小复现。）

6.9 落回 Transformer

```
scores = q @ k.transpose(-2, -1) / math.sqrt(d_k) # (B, h, n, n)
scores = scores + causal_mask[:, :, :n, :n]         # 先屏蔽
attn    = scores.softmax(dim=-1)                   # 后归一化, dim 必须是 -1
out     = attn @ v                                  # (B, h, n, d_v) ← 唯一的左乘
```

四行，四个知识点，分别来自第 5、6、3、2 章：

- 第 1 行： $\sqrt{d_k}$ 校准（第 5 章）
- 第 2 行：先屏蔽后归一化，形状靠广播（第 3、6 章）
- 第 3 行：`dim=-1` 意味着“每个 query 对所有 key 归一化”，选错不报错（第 3 章）
- 第 4 行：整个 Transformer 里唯一的左乘（第 2 章）

第二部的核心内容到此结束。 你现在能逐行解释注意力机制的每一个符号。

6.10 自测

1. 权重为 $(0, 1, 0)$ 时输出是什么？ $(1/3, 1/3, 1/3)$ 时呢？分别对应凸包上的什么位置？
2. 证明推论三：若 $a_j \geq 0$ 且 $\sum a_j = 1$ ，则 $\|\sum a_j v_j\| \leq \max \|v_j\|$ 。（提示：三角不等式加齐次性。）如果去掉“和为 1”的条件，结论还成立吗？
3. 某人把因果掩码实现成“softmax 之后把上三角置零”。（a）每行权重和会变成什么？（b）第 0 个 token 和第 100 个 token 受到的影响一样大吗？（c）这个 bug 会让程序崩溃吗？
4. `attn` 是 (n, n) ，`v` 是 $(n, 64)$ ， $n = 4096$ 。`out` 的秩最多是多少？如果 $n = 100$ 呢？用一句话说明这个上界对单个头的表达力意味着什么。
5. 列出 softmax 在注意力中承担的三个角色，并各举一个“如果去掉它会发生什么”的后果。

6. (思考题) 有人提议把 softmax 换成逐元素的 sigmoid (每个权重独立地压到 0~1, 但不做归一化)。(a) 输出还是凸组合吗? (b) 推论三还成立吗? (c) 注意力沉降现象 (第 5 章) 还会以同样的形式出现吗?

本章小结

- $\text{out 第 } i \text{ 行} = \sum_j \text{attn}[i][j] \cdot (V \text{ 第 } j \text{ 行})$, 这是第 2 章线性组合视角的直接应用。
- softmax 保证系数非负且和为一, 因此输出是 value 向量的凸组合, 落在它们的凸包内。
- 三个推论: 不创造新方向 (只插值)、秩 $\leq \min(n, d_v)$ (第 7 章正题)、范数不超过最长的 value (免费的稳定性)。
- softmax 的三重角色: 切断结合律、提供非线性、保证凸组合。
- 因果掩码必须在 softmax 之前加 $-\infty$ 。先屏蔽后归一化, 顺序颠倒会破坏权重和为 1。
- 注意力负责通信, FFN 负责计算。

本章刻意没讲

凸集与凸函数的一般理论、凸包的计算方法、单纯形与重心坐标、Jensen 不等式、最优传输视角下的注意力。

本章只需要"非负且和为一 $\implies$ 落在凸包内"这一条事实。它的完整理论体系与本书主线无关。

下一章预告

第 6 章留下了一个数字: 单个头的输出, 秩最多 64。

第 7 章会追问下去。你会看到一个更尖锐的版本: 真正被学习的 $W_q W_k^T$ 是一个 768×768 的矩阵, 但它的秩最多只有 64——注意力的打分能力从一开始就被封在一个低秩的盒子里。

这就是多头注意力存在的数学理由。同一个洞察还能直接解释 LoRA 为什么可行、以及它的能力边界在哪。第 2 章那个被搁置的"外积求和视角", 将在这一章派上唯一但关键的用场。

第 7 章结束后, 第二部完成, 注意力机制就被彻底拆开了。

第 7 章 秩与低秩瓶颈

本章要回答：12 个 64 维的头，和 1 个 768 维的头，参数量完全一样。为什么前者更好？

读完本章你能做到：说清秩是什么以及为什么它衡量"表达力"；算出注意力打分矩阵的秩上界并解释其后果；用同一个洞察解释 LoRA 的可行性与能力边界。

本章是第二部技术密度最高的一章。第 2 章那个被搁置的"外积求和视角"，将在这里派上唯一但决定性的用场。

7.1 困惑：参数量一样，为什么要拆

先把账算清楚。

方案 A：单头， $d_k = 768$ 。 需要 $W_q \in \mathbb{R}^{768 \times 768}$ 和 $W_k \in \mathbb{R}^{768 \times 768}$ ，共 2×768^2 个参数。

方案 B：12 个头，每头 $d_k = 64$ 。 每个头需要 $2 \times 768 \times 64$ ，12 个头合计 $2 \times 768 \times 768$ 。

完全相同。 多头不省一个参数。

那为什么全世界的模型都选 B？常见的解释是"不同的头关注不同的方面"——这话没错，但它是描述，不是理由。**方案 A 的单个头也可以同时关注很多方面，它的容量还更大。**

要真正回答这个问题，需要秩。而答案会分成两半，很多材料把它们混为一谈了。

7.2 最小必要定义：秩

本书用第 2 章的外积视角来定义秩，因为它最直接。

先看什么是**秩 1 矩阵**：一个列向量乘一个行向量得到的矩阵。

$$\begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \otimes (1, 1, 2) = \begin{bmatrix} 2 & 2 & 4 \\ 1 & 1 & 2 \\ 3 & 3 & 6 \end{bmatrix}$$

特征很明显：**所有行都互相成比例**（第 2 行 $\times 2 =$ 第 1 行， $\times 3 =$ 第 3 行）。它看起来是 3×3 有 9 个数，实际的自由度只有 $3+3-1=5$ 个。

秩：把矩阵写成若干个秩 1 矩阵之和时，所需的**最少项数**。

等价的说法（本书不证明）：秩 = 线性无关的行数 = 线性无关的列数。

秩衡量的是矩阵真正的自由度。一个 768×768 的矩阵有 589824 个数，但如果它的秩只有 64，那它实际能表达的东西远远少于形状暗示的量——就像一张 4K 分辨率的图片，如果内容只是一片纯色，它承载的信息和 1 个像素一样多。

核心不等式

$\text{rank}(AB) \leq \min(\text{rank } A, \text{rank } B) \leq \text{中间那个消失的维度}$

用外积视角一眼就能看出来。第 2 章讲过：

$$AB = \sum_{k=1}^b (A \text{ 的第 } k \text{ 列}) \otimes (B \text{ 的第 } k \text{ 行})$$

这是 **b 个秩 1 矩阵之和**，而 b 就是相乘时消失的那个中间维度。既然 b 项就够了，最少项数当然不会超过 b。

一个矩阵乘法的秩，被它最窄的那个"腰"卡住。

这句话是本章剩下所有内容的唯一工具。

7.3 注意力的低秩瓶颈

回到第 5 章的那个式子：真正被学习的是

$$\text{scores}[i][j] = x_i (W_q W_k^T) x_j^T$$

看中间的 $M = W_q W_k^T$ ：

$$\begin{array}{c} (768 \times 64) @ (64 \times 768) \rightarrow (768 \times 768) \\ \uparrow \\ \text{腰} = 64 \end{array}$$

M 是一个 768×768 的矩阵，但

$$\text{rank}(\mathbf{M}) \leq 64$$

它的形状暗示有 589824 个自由度，实际只有约 $2 \times 768 \times 64 \approx 98304$ 个——六分之一。

单个注意力头能表达的"比较标准"，从一开始就被封在一个低秩的盒子里。

更尖锐的版本：分数矩阵本身也是低秩的

同样的腰卡在分数矩阵上：

$$\mathbf{S} = \mathbf{Q} \mathbf{K}^T = (n \times 64) @ (64 \times n) \rightarrow (n \times n)$$

于是：

$$\text{rank}(\mathbf{S}) \leq d_k = 64, \text{ 与 } n \text{ 无关}$$

$n = 4096$ 时，这个 4096×4096 的分数矩阵，秩最多 64。

这个数字值得停一下。它意味着：在 4096 个 token 之间，单个头能表达的注意力模式，其自由度只有 64 维。绝大多数你能想象出来的 4096×4096 分数矩阵，这个头根本表达不出来。

序列越长，这个瓶颈越紧——因为 n 在涨，而上界纹丝不动。

缓解因素：softmax 会打破低秩

必须补一句，否则会误导。

$\mathbf{S}$ 是低秩的，但注意力权重是 $\text{softmax}(\mathbf{S})$ 。 softmax 里有 $\exp$ ，是逐元素的非线性，它会破坏低秩结构。一个秩 64 的矩阵取指数之后，秩通常会大幅上升。

所以真实情况是：打分的自由度被限制在 64 维，但经过 softmax 之后得到的权重矩阵可以是高秩的。瓶颈是真实的，但没有 $\text{rank}(\mathbf{S}) \leq 64$ 听上去那么绝望。

7.7 节的实验会让你亲眼看到这两件事同时发生。

7.4 多头的真正理由：两个独立的论点

现在可以回答 7.1 了。答案有两半，请分开记。

论点一：d_k 不能太小

低秩瓶颈说明 d_k 是打分表达力的硬约束。d_k = 64 已经不宽裕，如果一味增加头数把 d_k 压到 8 或 4，每个头的分数矩阵秩就只有 8 或 4，注意力模式会退化到极其粗糙。

极端情况：h = 768，每头 d_k = 1。此时每个头的分数矩阵是秩 1 的——所有行成比例，意味着所有 query 的注意力偏好只差一个缩放因子。这样的头几乎无法表达任何“因位置而异”的关注模式。

所以论点一是一条下界：d_k 不能太小。它解释了为什么头数不能无限增加。

论点二：一次 softmax 只能表达一种混合方案

但论点一不能解释为什么不用方案 A（单头 d_k = 768）。方案 A 的秩上界是 768，比多头的每个头都高，参数量还一样。它哪里不好？

关键在这里：

单头只能产生一个注意力分布，因此只能做一次加权平均。

第 6 章讲过，注意力输出是 value 的凸组合，混合比例由那一行权重决定。一个头，一行权重，一种混合方案。

假设第 5 个 token 同时需要两件事：从第 2 个 token 那里取语法信息，从第 40 个 token 那里取指代对象。单个 softmax 分布必须在这两者之间分配一个固定比例，然后把它们揉进同一个加权平均里——两份信息混在一起，下游无法再分开。

而 12 个头可以：头 A 全力关注位置 2，头 B 全力关注位置 40，两份结果被 concat 到不同的坐标段上，互不污染，然后由 W_o 决定怎么用。

多头的价值不在于打分的秩更高（它更低），而在于能同时执行 h 个独立的混合，并保持结果可分离。

这才是方案 B 胜出的真正原因。参数量相同，但并行的混合通道数从 1 变成了 12。

（经验证据也支持这个图景：研究者发现训练好的模型中许多头可以被剪掉而影响不大，说明冗余存在；但把头数降到很少时性能会明显下降，说明“多通道”这件事本身是必要的。）

7.5 W_o：每个头独立写入

多头结果拼接之后要过一个输出投影：

```
out = concat(head_1, ..., head_h) @ W_o      # (n, h · d_v) @ (h · d_v, d) → (n, d)
```

这一步常被当作"把维度变回去"的杂事，但用第2章的分块视角看，它有清晰的结构。

把 W_o 按行切成 h 块，每块 (d_v, d) ：

$$W_o = \begin{bmatrix} W_o^{(1)} \\ W_o^{(2)} \\ \vdots \\ W_o^{(h)} \end{bmatrix} \quad \text{每块 } d_v \text{ 行}$$

因为 `concat` 是沿列方向拼的，而切分是沿行方向做的，两者恰好对齐，于是：

$$\text{out} = \sum_{i=1}^h \text{head}_i @ W_o^{(i)}$$

多头的输出不是"拼在一起"，而是 h 个独立贡献的求和。

每个头计算自己的 head_i ，用自己那一块 $W_o^{(i)}$ 决定往哪个方向写，然后所有头的贡献直接相加。**头与头之间从头到尾没有任何交互。**

这个分解是可解释性研究的标准起点（每个头可以被单独分析、单独消融），也直接引向下一章：既然每个头的贡献是加到一起的，那么"加法"本身就成了模型的核心结构。第8章的主题正是这个。

7.6 反过来用：LoRA

低秩是注意力的限制，但也可以是主动的设计工具。

微调一个大模型，本来要更新 $W \in \mathbb{R}^{d \times d}$ ，即 d^2 个参数。LoRA 的做法是强制更新量为低秩：

$$W' = W + \Delta W, \quad \Delta W = A B, \quad A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times d}, \quad r \ll d$$

由核心不等式， $\text{rank}(\Delta W) \leq r$ 。

账： $d = 4096$, $r = 8$ 。

	参数量
全量微调一个矩阵	$4096^2 = 16,777,216$
LoRA	$2 \times 4096 \times 8 = 65,536$
比值	约 256 倍

它为什么可行？依赖一个经验假设：

把一个预训练模型适配到某个下游任务，所需的权重改变**本质上是低秩的**。

这不是定理，是观察。直觉上讲得通：预训练已经学会了绝大部分通用能力，微调只需要在少数方向上做调整。实践中在很多任务上确实成立。

它的能力边界也由同一个不等式给出：LoRA 无法表达秩超过 r 的改变。如果某个任务真的需要大范围重塑权重， $r = 8$ 就不够，必须调大 r 或改用全量微调。

同一条 $\text{rank}(AB) \leq \text{腰}$ ，在注意力里是需要绕开的限制，在 LoRA 里是刻意施加的约束。

7.7 动手

```
import numpy as np
rng = np.random.default_rng(0)
d, h = 768, 12
d_k = d // h                                # 64
Wq, Wk = rng.normal(size=(d, d_k)), rng.normal(size=(d, d_k))
M = Wq @ Wk.T
print(M.shape, np.linalg.matrix_rank(M))    # (768, 768) 秩 64

n = 512
X = rng.normal(size=(n, d))
S = (X @ Wq) @ (X @ Wk).T
print(S.shape, np.linalg.matrix_rank(S))    # (512, 512) 秩 64

P = np.exp(S - S.max(1, keepdims=True))
P /= P.sum(1, keepdims=True)
print(np.linalg.matrix_rank(P))             # 远大于 64: softmax 打破了低秩
```

建议做的三个实验：

1. 看见腰。把 d_k 依次设成 4、16、64、256，每次打印 `matrix_rank(Wq @ Wk.T)`。确认秩总是等于 d_k ，与 768 无关。
2. 看见 softmax 的解救（本章必做）。上面最后两行打印出两个秩：softmax 前是 64，softmax 后大得多。把 S 整体乘以 0.01 再做一次（相当于分数很小、softmax 接近线性），观察秩的变化。**你会看到 softmax 越接近线性，低秩瓶颈就越紧**。这解释了 7.3 节末尾那句“瓶颈真实但没那么绝望”的确切含义。
3. 验证 W_o 的分块分解。造 $h=4$ 个随机的 head_i （每个 $(n, 64)$ ）和一个 W_o $((256, d))$ 。分别计算 `concat(heads) @ W_o` 和 `sum(head_i @ W_o[i*64:(i+1)*64])`，用 `np.allclose` 确认两者相等。

7.8 落回 Transformer

一张表总结本章的全部数字（GPT-2 small: $d = 768$, $h = 12$, $d_k = 64$ ）：

对象	形状	秩上界	来源
$W_q W_k^T$ （单头学到的比较标准）	768×768	64	腰 = d_k
QK^T （单头分数矩阵, $n = 4096$ ）	4096×4096	64	腰 = d_k
$\text{softmax}(QK^T)$	4096×4096	可以很高	$\exp$ 是非线性
单头输出 $\text{attn} @ V$	4096×64	64	腰 = d_v
LoRA 的 ΔW ($r = 8$)	768×768	8	腰 = r

第二部到此结束。 从第 4 章的投影、第 5 章的打分、第 6 章的加权平均，到第 7 章的秩，注意力机制的每一个部件都已经被拆开看过。

7.9 自测

- （a）秩 1 矩阵有什么可以一眼认出的特征？（b）一个 100×100 的秩 1 矩阵，真正的自由度是多少个数？
- 用外积求和视角说明 $\text{rank}(AB) \leq \text{中间维}$ 。然后回答： $W_1 \in \mathbb{R}^{768 \times 64}$ 、 $W_2 \in \mathbb{R}^{64 \times 768}$ ， $W_1 W_2$ 的秩上界是多少？（这道题在第 4 章出现过，现在你应该能给出理由而不只是答案。）
- 某模型 $n = 8192$, $d = 4096$, $h = 32$ 。（a）单头分数矩阵的形状和秩上界分别是多少？（b）这个上界随 n 增长吗？（c）如果不考虑 softmax ，这意味着模型能表达的注意力模式受到多严重的限制？
- 比较两个方案的参数量与能力：（a）1 个头， $d_k = 768$ ；（b）12 个头， $d_k = 64$ 。分别回答：参数量、单个分数矩阵的秩上界、能同时产生几个不同的注意力分布、输出能否保持可分离。据此说明为什么实践中选 (b)。
- $h = 768$ 、 $d_k = 1$ 的极端情况：（a）单头分数矩阵的秩是多少？（b）"所有行成比例"在注意力语义上意味着什么？（c）由此说明头数为什么不能无限增加。
- LoRA, $d = 4096$, $r = 16$ 。（a）参数量比全量微调省多少倍？（b） ΔW 的秩上界？（c）如果某任务效果不佳，把 r 从 16 调到 64，参数量和秩上界各变成多少？
- 证明 W_o 的分块分解 $\text{out} = \sum_i \text{head}_i @ W_o^{(i)}$ 。（提示：concat 沿列、切分沿行，写出矩阵乘法的定义即可。）说明这个结论为什么让"逐个头做消融实验"成为可能。

本章小结

- 秩 = 写成秩 1 矩阵之和所需的最少项数 = 矩阵真正的自由度。

- 核心不等式： $\text{rank}(AB) \leq \text{腰}$ （相乘时消失的那个中间维）。本章全部结论都从这一条推出。
- 单头的 $W_q W_k^T$ 是 768×768 但秩 ≤ 64 ；分数矩阵 QK^T 是 $n \times n$ 但秩 ≤ 64 ，与 n 无关。
- softmax 的非线性会打破低秩，缓解但不消除这个瓶颈。
- 多头的理由有两个，必须分开： d_k 不能太小（低秩瓶颈给出下界）；一次 softmax 只能表达一种混合方案（多头提供并行的混合通道，且结果保持可分离）。后者才是多头胜过单个大头的理由。
- W_o 可按行分块，于是 $\text{out} = \sum_i \text{head}_i @ W_o^{\omega}$ ——多头输出是 h 个独立贡献之和，头与头之间无交互。
- LoRA 把同一条不等式反过来用：主动约束 ΔW 的秩，用 $2r$ 个参数近似 d^2 个，代价是无法表达高秩改变。

本章刻意没讲

奇异值分解、秩的严格证明（行秩=列秩）、秩-零化度定理、矩阵的数值秩与阈值选择、低秩逼近的最优性（Eckart-Young 定理）。

本书只需要"腰卡住秩"这一条不等式。SVD 是它的精确化版本，放在附录 A——那里它服务于另一个问题（训练稳定性），与本章的主线无关。

第二部结束

你现在能逐符号解释 $\text{softmax}(QK^T/\sqrt{d_k})V$ ，并说清每个设计选择的理由。

第三部转向一个不同的视角。前七章都在看"数据怎么被变换"，接下来三章看**模型在几何上是什么样子**：

- **第8章**：残差流。本章末尾那个" h 个贡献相加"的结构会被推广到整个模型——你会看到 Transformer 的主干其实是一条贯穿所有层的加法总线，而每一层只是往里读写。这一章还会正面处理第1章埋下的最大伏笔：**768 维为什么装得下远超 768 个概念**。
- 第9章：LayerNorm 的几何含义。
- 第10章：正交矩阵与 RoPE。

第三部是全书唯一带有审美性质的部分，也是读完之后最容易长期记住的部分。

第 8 章 加法、残差流与高维容量

本章要回答：768 维的空间，凭什么装得下几万个概念？

读完本章你能做到：把整个模型重写成一条加法总线；解释残差连接为什么让深度网络可训练；给出“高维空间能容纳的近似正交方向数远超维数”的定量论证。

本章兑现全书最大的一个伏笔（第 1 章 1.5 节埋下）。它也是全书唯一一章，读完之后你会觉得高维空间本身是个有点神奇的地方。

8.1 困惑：容量对不上

第 7 章末尾留下了一个结构性观察：多头输出是 h 个独立贡献之和。

$$\text{out} = \sum_i \text{head}_i @ W_o^{(i)}$$

这不是巧合。放大来看，整个 Transformer 的主干都是加法。

而加法带来一个立刻的困惑。一个 768 维空间里，最多只能放 768 个互相正交的方向。但一个语言模型需要表示的东西远不止 768 个——词汇、语法角色、指代关系、事实知识、语气、语言种类……保守估计也要成千上万。

768 个坑，几万件东西。怎么放得下？

这个问题的答案会改变你对高维空间的直觉。

8.2 残差流：一条贯穿全模型的总线

每个子层的形式都是：

$$x \leftarrow x + f(x)$$

注意力和 FFN 都不是“替换” x ，而是往 x 上加东西。把 L 层展开：

$$\mathbf{x}_L = \mathbf{x}_0 + \sum_{l=1}^L \mathbf{f}_l(\dots)$$

最终的向量 = 初始嵌入 + 每一层贡献的总和。

这条从头贯穿到尾、被所有层共享的 768 维向量，通常被称为**残差流**（residual stream）。

第 3 章的形状表其实已经把这件事画出来了：一层的输入是 $(B, n, 768)$ ，输出还是 $(B, n, 768)$ ，中间十几步的形变最后都收回原形。**输入输出必须同形，加法才能成立**——这不是巧合，是残差结构的硬性要求。

于是模型的图景变了。它不是一条“数据被逐层加工”的流水线，而更像一块共享白板：

残差流是总线。每一层从上面读一点东西，算一算，再把结果写回去。

8.3 读和写，都是线性代数

“读”和“写”不是比喻，它们各自对应一个明确的运算。

读 = 右乘一个矩阵

第 4 章讲过， $\mathbf{x} @ \mathbf{W}$ 是把 $\mathbf{x}$ 投影到某个子空间——只保留 $\mathbf{W}$ 关心的那些方向。

- 注意力头用 $\mathbf{W}_q$ 、 $\mathbf{W}_k$ 读取残差流中与“该关注谁”有关的部分；
- FFN 用 $\mathbf{W}_1$ 读取与“该触发哪些模式”有关的部分。

每个组件都只读它需要的那几十个方向，对残差流的其余部分视而不见。

写 = 加上一个向量

而写入的方向由输出矩阵的行决定。用第 2 章的线性组合视角：

$$\text{head} @ \mathbf{W}_o^{(i)} = \sum_k \text{head}_k \cdot (\mathbf{W}_o^{(i)} \text{ 的第 } k \text{ 行})$$

$\mathbf{W}_o^{(i)}$ 的每一行是一个候选写入方向，head 的分量决定各写入多少。

FFN 完全同理： $\mathbf{W}_2$ 的每一行是一个候选写入方向，激活值决定系数。第 11 章会展开。

于是整个模型可以这样重写

残差流：一个 768 维向量
 每一层：读几十个方向 $\rightarrow$ 算 $\rightarrow$ 往几十个方向写
 最终：所有写入累加，过 unembedding 得到 logits

没有别的了。全部 96 层、几十亿参数，做的就是同一个 768 维向量上反复读写。

这个视角有一个直接的实践后果：因为贡献是**相加**的，你可以把任意一个头或任意一层的贡献单独减掉，看输出怎么变。消融实验之所以有意义，正是因为加法可分解。（如果是乘法或复杂嵌套，"去掉某一部分"根本无法良好定义。）

8.4 顺带解决的问题：梯度高速公路

残差连接最初不是为"总线"这个图景发明的，而是为了让深网络能训起来。用线性代数看一眼原因。

把 $x + f(x)$ 写成 $x @ I + f(x)$ ， I 是单位矩阵——一个"什么都不做"的线性变换。求导：

$$\partial x_{\{l+1\}} / \partial x_l = I + \partial f / \partial x$$

永远有一个 I 在那里。反向传播时，梯度沿 L 层相乘：

$$\prod_{\{l\}} (I + J_l)$$

展开后包含一条纯粹由 I 组成的路径——梯度可以**原封不动**地从最后一层传到第一层。

对比没有残差的情况：

$$\prod_{\{l\}} W_l$$

这是一串矩阵连乘。如果每个 W 略微放大， $L=96$ 层之后就爆炸；略微缩小，就消失。这正是深度网络在 ResNet 之前难以训练的原因。

加法给了梯度一条不受阻碍的通路。

（顺带说，这也是"残差"这个名字的由来： $f(x)$ 学的不是完整的输出，而是相对于恒等映射的**修正量**。当某一层没什么可做时，它只需要输出接近 0，而不是费力地学出一个单位矩阵。）

8.5 核心：高维空间有多慷慨

现在回到 8.1 的容量问题。

第一步：放弃"严格正交"这个要求。

两个方向要"互不干扰"，不必严格垂直。只要夹角足够接近 90°、内积足够小，读取其中一个时另一个造成的干扰就可以忽略。

第二步：看看随机方向有多接近正交。

在 $\mathbb{R}^d$ 中随机取两个单位向量，它们内积的分布是：

均值 = 0，标准差 $\approx 1/\sqrt{d}$

$d = 768$ 时，标准差约为 0.036。换句话说，两个随机的 768 维方向，夹角几乎总是在 88° 到 92° 之间。

这已经很反直觉了。在二维平面上随手画两条线，夹角可以是任意值；在 768 维里随手扔两个方向，它们几乎必然近似垂直。高维空间里"随便一放就是正交的"。

第三步：算一算能放多少。

关键问题是：扔 N 个随机方向，其中最"挤"的那一对有多挤？

N 个方向有约 $N^2/2$ 对，每对的内积是标准差 $\sigma = 1/\sqrt{d}$ 的随机数。 M 个这样的随机数中最大值约为 $\sigma \sqrt{2 \ln M}$ 。代进去：

方向数 N	最大两两 $ \cos $
1,000	≈ 0.19
1,000,000	≈ 0.27
10,000,000,000	≈ 0.34

请看这张表最惊人的地方：方向数增加一万倍，最大干扰只从 0.27 涨到 0.34。

原因在公式里：干扰随 $\sqrt{(\ln N)}$ 增长，而这是一个极其缓慢的函数。指数级地增加方向数，只需付出对数的平方根的代价。

结论：768 维空间可以容纳数以亿计的近似正交方向。

" d 维最多 d 个方向"这个直觉，只在要求严格正交时才成立。放松到"近似正交"，容量立刻从线性变成指数。

第四步：这就是叠加。

模型确实在利用这一点。它把远多于 768 个特征，存放在 768 维空间的不同近似正交方向上。这个现象被称为**叠加**（superposition）。

代价是特征之间会互相干扰——但如你所见，干扰小得惊人。

8.6 干扰的代价，以及稀疏性为什么是前提

把叠加写清楚。设残差流是若干特征的叠加：

$$x = \sum_j a_j v_j \quad (v_j \text{ 是各特征的方向, } a_j \text{ 是激活强度})$$

某个组件想读出特征 i ，它用 v_i 做投影：

$$x \cdot v_i = \underbrace{a_i \|v_i\|^2}_{\text{想要的}} + \underbrace{\sum_{j \neq i} a_j (v_j \cdot v_i)}_{\text{干扰噪声}}$$

噪声项的大小取决于两件事：**每对方向的内积有多小**（约 0.03~0.3），以及**同时有多少特征被激活**。

如果 k 个特征同时激活、符号随机，噪声大致按 $\sqrt{k}$ 增长。所以：

叠加成立的前提是稀疏性——任何时刻只有少数特征被激活。

这个条件在语言上大致成立。处理一句关于烹饪的话时，关于量子力学、法语语法、股票代码的特征基本都是零。**是稀疏性买来了容量。**

两个直接后果

多义神经元。 既然特征方向不与坐标轴对齐，那么任何单个坐标（“神经元”）都会同时参与多个特征。所以你看某个神经元什么时候激活，常常得到一份毫无关联的清单——它不是坏了，它只是从来就不代表单一概念。

稀疏自编码器（SAE）在做什么。 既然特征以近似正交方向叠加存储，就可以训练一个模型把它们**解开**：把 768 维残差流映到一个远高维（比如 65536 维）但稀疏的空间，让每个维度对应一个真正单一的特征。这是目前可解释性研究的主流路线，而它的全部理论依据就是本节这几行。

8.7 带宽是有限的

反过来说，残差流的 768 维是**所有层共享的带宽**。

层数越多、每层写入的东西越多，这条总线就越拥挤，干扰噪声就越大。这给出一个真实的架构张力：

模型	d	层数 L	d / L
GPT-2 small	768	12	64
GPT-2 XL	1600	48	33
Llama-3-8B	4096	32	128

单纯堆层数而不加宽 d，会让每一层能"独占"的带宽越来越少。实践中 d 和 L 大致同步增长，不是随意的选择。

8.8 动手

```
import numpy as np
rng = np.random.default_rng(0)

def stats(d, N=2000):
    V = rng.normal(size=(N, d))
    V /= np.linalg.norm(V, axis=1, keepdims=True)
    C = V @ V.T
    np.fill_diagonal(C, 0.0)
    return C.std(), np.abs(C).max()

for d in [8, 64, 768, 4096]:
    s, m = stats(d)
    print(f"d={d:5d} 实测std={s:.4f} 理论1/√d={1/np.sqrt(d):.4f} 最大|cos|={m:.3f}")
```

你会看到实测标准差和 $1/\sqrt{d}$ 几乎完全重合，而 2000 个方向在 768 维里的最大 $|\cos|$ 只有 0.2 上下。

建议做的三个实验：

1. 看见"随便一放就正交"（本章必做）。运行上面的代码，然后把 N 从 2000 加到 20000，观察最大 $|\cos|$ 变化了多少。方向数翻十倍，干扰几乎没涨——这就是 $\sqrt{\ln N}$ 的威力，比读表格有说服力得多。
2. 测量干扰噪声。在 768 维里造 5000 个随机单位向量当作"特征"。随机选 k 个叠加成 x（系数取 1），然后用某个被选中的 v_i 去读：`x @ v_i`。理想值是 1，看实际读到多少。把 k 依次取 1、5、20、100，观察噪声如何随 $\sqrt{k}$ 增长。这个实验能让你亲手看到"稀疏性买来容量"。
3. 验证残差的可加性。造一个初始向量和 12 个"层贡献"，求和得到输出。然后去掉第 5 个贡献重算，确认差值精确等于那个贡献本身。再想一想：如果模型是纯粹的层层相乘，"去掉第 5 层"这件事还能这样定义吗？

8.9 落回 Transformer

```
for block in self.blocks:
    x = x + block.attn(block.ln1(x))    # 读 → 算 → 写回
    x = x + block.mlp(block.ln2(x))    # 读 → 算 → 写回
logits = self.ln_f(x) @ self.W_u
```

两个 $x = x + \dots$ 就是整个模型的主干。其余所有代码，都是在描述那两个加号右边该加什么。

这个结构还解释了一个有趣的实验事实：把中间某一层的残差流直接乘上 unembedding 矩阵，往往能得到有意义的词分布——而且越靠近顶层越准。这被称为 logit lens。

它之所以能work，正是因为残差流从头到尾住在同一个空间里，每层只是往上叠加修正。第 5 层的 x 不是"中间表示"，它已经是一个可以直接解读的、只是还没修完的预测。

8.10 自测

1. 把 $x \leftarrow x + f(x)$ 展开 L 层，写出 x_L 关于 x_0 和各层 f 的表达式。据此说明：为什么"消融第 7 层"这个操作可以被精确定义？
2. (a) 求 $\partial(x + f(x))/\partial x$ 。(b) L 层连乘之后，为什么梯度不会指数消失？(c) 如果去掉残差连接，96 层的乘积链会出什么问题？
3. 在 $\mathbb{R}^d$ 中随机取两个单位向量。(a) 内积的均值和标准差各是多少？(b) $d = 4096$ 时标准差是多少？(c) "高维空间里随便两个方向几乎必然近似垂直"——这句话在 $d = 2$ 时成立吗？在 $d = 10$ 呢？
4. 最大两两 $|\cos|$ 随方向数 N 按 $\sqrt{\ln N}$ 增长。(a) N 从 10^3 增到 10^6 ，干扰增加多少倍？(b) 从 10^6 增到 10^{12} 呢？(c) 用一句话说明为什么"d 维最多 d 个方向"这个直觉是错的。
5. 残差流是 5000 个特征的叠加，两两 $|\cos| \approx 0.05$ 。(a) 若同时有 4 个特征激活（系数均为 1），读取其中一个时噪声大约多大？(b) 若同时有 400 个呢？(c) 由此说明稀疏性为什么是叠加的必要条件。
6. 用叠加解释"多义神经元"：为什么单个坐标的激活模式看起来毫无规律？这是模型的缺陷还是必然结果？
7. (思考题) 某人想把模型层数从 32 加到 128，同时保持 $d = 4096$ 不变。从残差流带宽的角度，可能出现什么问题？

本章小结

- $x_L = x_0 + \sum_l f_l(\dots)$ ：模型的主干是一条贯穿所有层的加法总线，即残差流。层的输入输出必须同形，加法才成立。
- 读 = 右乘矩阵（投影到子空间）；写 = 加上一个向量。写入方向由 W_o 、 W_2 的行给出。
- 加法让梯度多出一条 I 路径， $\Pi(I + J_l)$ 中永远存在恒等通路——这是深度可训练的原因。

- 加法还让贡献可分解，消融实验因此有意义。
- 在 $\mathbb{R}^d$ 中随机两个单位向量的内积，均值 0、标准差 $1/\sqrt{d}$ 。 $d = 768$ 时约 0.036，即随手一放就近似正交。
- 最大干扰按 $\sqrt{\ln N}$ 增长，所以方向数可以指数级增加而代价极小。**768 维能容纳数以亿计的近似正交方向。**
- 这就是叠加：特征数 $\gg$ 维数，代价是轻微干扰，前提是**稀疏激活**。
- 推论：多义神经元是必然而非缺陷；稀疏自编码器就是把叠加解开的工具。
- 残差流是共享带宽， d 与层数需要同步增长。

本章刻意没讲

测度集中现象的证明、高维球面的体积分布、Kabatiansky–Levenshtein 等球堆积界、叠加假说的完整实验证据、稀疏自编码器的训练细节。

本章只需要两个数字： $1/\sqrt{d}$ 和 $\sqrt{\ln N}$ 。它们已经足以支撑全部结论，而且都可以在自己的电脑上五分钟验证。

下一章预告

残差流是一条被反复读写的总线。但如果每一层都往上加东西，向量的长度会怎样？

第 9 章处理这个问题。你会看到 LayerNorm 在几何上做了一件非常具体的事：**先把向量投影到一个超平面上，再把它推到一个球面上**。两步都能用一行矩阵表达。

理解了这个几何图像，你会立刻明白 RMSNorm 为什么可以省掉其中一步而效果几乎不变——这是本书里"几何直觉直接解释工程选择"的最好例子。

第9章 归一化的几何

本章要回答：减均值、除标准差，这个统计操作在几何上是什么？

读完本章你能做到：把 LayerNorm 写成"一次投影 + 一次缩放"；证明它的输出范数恒为 $\sqrt{d}$ ；用几何直觉解释 RMSNorm 为什么能省掉一半却几乎不掉点。

本章是全书"几何直觉直接解释工程选择"的最好例子。看懂两步几何之后，RMSNorm 的合理性不需要任何额外说明。

9.1 困惑：一个统计操作，混在一堆矩阵里

第8章说，残差流是一条总线，每层往上加东西。

那么长度会怎样？如果96层每层都加一个向量，而这些向量近似正交（第8章告诉我们它们通常是），那么范数大致按 $\sqrt{L}$ 增长。深层的残差流会比浅层大好几倍，各层看到的输入尺度完全不同。

Transformer 的解决办法是归一化：

$$\text{LayerNorm}(x) = \gamma \odot (x - \mu) / \sigma + \beta, \quad \mu = (1/d) \sum x_i, \quad \sigma^2 = (1/d) \sum (x_i - \mu)^2$$

这个式子看起来像统计学，和前八章的矩阵世界格格不入。但它其实是两个纯粹的几何操作，每一个都能用一行线性代数写出来。

先澄清一个高频混淆：LayerNorm 是对单个 token 沿特征维度做的——它读取一个768维向量里的768个数，算它们的均值和方差。它不跨 token、不跨 batch。

这一点很关键。用第2章的框架说：LayerNorm 是逐行操作，不混合 token。如果换成 BatchNorm（跨样本统计），自回归模型就完蛋了——预测第5个 token 时会用到第50个 token 的统计量，因果性直接破裂。

9.2 第一步：减均值 = 投影到超平面

设 $\mathbf{1} = (1, 1, \dots, 1)$ 是全1向量。均值可以写成内积：

$$\mu = (1/d) \cdot (x \cdot \mathbf{1})$$

于是"每个分量都减去 μ "就是

$$x - \mu \cdot \mathbf{1} = x - (1/d)(x \cdot \mathbf{1}) \cdot \mathbf{1} = x (I - (1/d) \cdot \mathbf{1} \mathbf{1}^T)$$

记 $P = I - (1/d) \cdot \mathbf{1} \mathbf{1}^T$ 。这是一个 $d \times d$ 的矩阵，减均值就是右乘它。

P 是一个投影矩阵，两条性质可以直接验证：

- 对称： $P^T = P$ 。
- 幂等： $P^2 = P$ 。（投影两次等于投影一次——已经压平的东西再压一次不会变。）

它投到哪里？看结果和 $\mathbf{1}$ 的内积：

$$(x - \mu \mathbf{1}) \cdot \mathbf{1} = (x \cdot \mathbf{1}) - \mu \cdot d = (x \cdot \mathbf{1}) - (x \cdot \mathbf{1}) = 0$$

输出永远垂直于全 1 方向。

减均值 = 把向量投影到"与全 1 向量垂直"的那个 $(d-1)$ 维超平面上。

沿 $\mathbf{1}$ 方向的那一个分量被彻底丢弃了。 P 的秩是 $d-1$ ，被压掉的正是那 1 维。

9.3 第二步：除以标准差 = 推到球面上

减完均值后，标准差和范数是同一件事：

$$\sigma^2 = (1/d) \sum (x_i - \mu)^2 = \|x - \mu \mathbf{1}\|^2 / d \quad \rightarrow \quad \sigma = \|x - \mu \mathbf{1}\| / \sqrt{d}$$

代进去：

$$\hat{x} = (x - \mu \mathbf{1}) / \sigma = \sqrt{d} \cdot (x - \mu \mathbf{1}) / \|x - \mu \mathbf{1}\|$$

右边是单位向量乘以 $\sqrt{d}$ 。所以：

$\| \text{LayerNorm 的输出} \| = \sqrt{d}$ ，恒定不变。

无论输入是什么，输出的长度永远是 $\sqrt{d}$ 。 $d = 768$ 时约 27.7。

两步合起来

LayerNorm 把任意向量映到：(d-1) 维超平面 $\cap$ 半径 $\sqrt{d}$ 的球面。

这个交集是一个球面（严格说是 (d-2) 维的球面，因为它是 (d-1) 维空间里的一个球面）。输入是 d 维的，输出只剩 d-2 个自由度。

丢掉的两个自由度是：沿全 1 方向的分量（整体平移），以及向量的长度（整体缩放）。

γ 和 β 还原了什么

紧接着的 $\gamma \odot \hat{x} + \beta$ 把这两个自由度还了回来——但要注意还的是什么。

γ 和 β 是每层固定的可学习参数，不随输入变化。它们能设定"这一层希望的尺度和偏移"，但无法恢复某个具体输入原本的 μ 和 σ 。那两个数确实被永久丢弃了。

归一化不是"标准化后再还原"，而是"强制丢弃两个自由度，然后由模型统一指定替代值"。

9.4 RMSNorm：省掉第一步

近年的模型（LLaMA、PaLM 等）大量改用 RMSNorm：

$$\text{RMSNorm}(x) = \gamma \odot \sqrt{d} \cdot x / \|x\|$$

它不减均值，直接缩放到球面。几何上说：只做第二步，不做第一步；丢掉 1 个自由度而不是 2 个。

代码上省掉一次求均值和一次减法，实测能快 10% 上下。**但为什么效果几乎不掉？**

答案是第 8 章的一个直接推论。

减均值丢弃的是 x 沿全 1 方向的分量。这个分量有多大？就是 $\cos(x, \mathbf{1})$ 的大小。而第 8 章告诉我们：

在 $\mathbb{R}^d$ 中，一个向量与任意固定方向的夹角余弦，标准差约为 $1/\sqrt{d}$ 。

$d = 4096$ 时，这个值约 **0.016**。也就是说，**残差流本来就几乎垂直于全 1 方向**——第一步基本上什么也没做，它在把一个已经接近 0 的分量减掉。

高维空间里，减均值这一步几乎是多余的。省掉它，也就没什么可省的。

这是本书里几何直觉最直接地兑换成工程决策的一处：不需要做消融实验，看一眼 $1/\sqrt{d}$ 就知道这一步不重要。

9.5 放在哪里：Pre-LN 与 Post-LN

归一化的位置有两种放法：

Post-LN (原始 Transformer) : $x \leftarrow \text{LN}(x + f(x))$
 Pre-LN (现代模型) : $x \leftarrow x + f(\text{LN}(x))$

差别看起来很小，实际影响很大。

Post-LN 把归一化压在残差流上。 每层的输出都被强制拉回球面，梯度回传时要穿过归一化，第 8 章那条干净的 I 通路被打断了。原始 Transformer 因此需要小心的学习率 warmup 才能训起来。

Pre-LN 只归一化"读出来的副本"。 用第 8 章的总线图景说：

读取总线时做校准，写回总线时不动它。残差流本身从头到尾没有被归一化过。

于是 $x = x + f(\text{LN}(x))$ 里那条 x 的通路完全没有被干扰，梯度可以原样穿过 96 层。这就是现代模型几乎一致选择 Pre-LN 的原因。

(代价是残差流的范数会随层数增长，所以 Pre-LN 模型通常在最后再加一个 ln_f ，把最终输出校准一次再送进 unembedding。第 8 章 8.9 节的代码里就有这一行。)

9.6 顺带：它是非线性的

第 4 章的表格把归一化列为模型的三处非线性之一。这里可以看清原因。

对 RMSNorm 而言，对任意 $\lambda > 0$ ：

$$\text{RMSNorm}(\lambda x) = \sqrt{d} \cdot (\lambda x) / \|\lambda x\| = \sqrt{d} \cdot x / \|x\| = \text{RMSNorm}(x)$$

它对输入的缩放完全免疫。一个线性函数满足 $f(\lambda x) = \lambda f(x)$ ，而这里是 $f(\lambda x) = f(x)$ ——不仅非线性，而且是尺度信息的彻底销毁者。

这个性质有个实用后果：归一化之后，前一层权重的整体缩放不再影响输出。这也是为什么带归一化的网络对权重初始化的尺度不那么敏感。

9.7 动手

```
import numpy as np
rng = np.random.default_rng(0)
d = 768
x = rng.normal(size=d) * 3 + 5          # 故意让均值非 0、方差非 1

ln = (x - x.mean()) / x.std()
print(np.linalg.norm(ln), np.sqrt(d))   # 27.71 27.71 ← 范数恒为  $\sqrt{d}$ 
print(ln.sum())                         #  $\approx 0$  ← 落在超平面上

u = np.ones(d) / np.sqrt(d)
P = np.eye(d) - np.outer(u, u)
print(np.allclose(P @ P, P))           # True ← 幂等，确实是投影
print(np.allclose(x @ P, x - x.mean())) # True ← 减均值就是右乘 P
print(np.linalg.matrix_rank(P))        # 767 ← 压掉了 1 维

rms = np.sqrt(d) * x / np.linalg.norm(x)
print(np.linalg.norm(rms))             # 27.71 ← 也在球面上
```

建议做的三个实验：

1. 看见"减均值几乎没用"（本章必做）。造一个纯随机的 `x = rng.normal(size=d)`（不加偏移），计算 `x.mean() * np.sqrt(d) / x.std()`——这就是它沿全 1 方向的归一化分量。把 `d` 依次取 8、64、768、4096，重复 1000 次取平均。你会看到它按 $1/\sqrt{d}$ 缩小。这一个实验就解释了 RMSNorm 为什么可行。
2. 比较两种归一化的输出差异。对同一批随机向量分别做 LayerNorm 和 RMSNorm，计算两者的余弦相似度。`d` 越大，越接近 1。
3. 验证尺度免疫。计算 `RMSNorm(x)` 和 `RMSNorm(100 * x)`，确认两者逐位相等。再对 LayerNorm 试一次——它对缩放也免疫吗？对平移呢？（两者答案不同，想清楚为什么。）

9.8 落回 Transformer

```
x = x + self.attn(self.ln1(x))          # Pre-LN: 归一化的是副本，不是流本身
x = x + self.mlp(self.ln2(x))
x = self.ln_f(x)                       # 最后统一校准一次
logits = x @ self.W_u
```

参数量上，归一化层小到几乎可以忽略：每个 LayerNorm 只有 $2d$ 个参数（ γ 和 β ），RMSNorm 只有 d 个。GPT-2 small 全部 25 个归一化层加起来不到 4 万参数，占总量的万分之三。

但它决定了模型能不能训起来。这是一个参数量与重要性完全不成比例的组件。

9.9 自测

1. $P = I - (1/d) \cdot \mathbf{1}\mathbf{1}^T$ 。(a) 验证 $P^2 = P$ 。(b) P 的秩是多少？(c) 哪个方向被 P 映成零向量？
2. 证明 $\| \text{LayerNorm}(x) \| = \sqrt{d}$ (不考虑 γ 、 β)。 $d = 4096$ 时这个值是多少？
3. LayerNorm 的输出有多少个自由度？丢掉的两个分别是什么？ γ 和 β 能把原始输入的 μ 和 σ 恢复回来吗？为什么？
4. (a) 一个随机的 d 维向量与全 1 方向的余弦，典型大小是多少？(b) $d = 4096$ 时是多少？(c) 据此论证 RMSNorm 为什么可以省掉减均值。
5. Pre-LN 与 Post-LN：(a) 分别写出公式。(b) 从第 8 章的梯度通路角度，说明为什么 Pre-LN 更容易训练。(c) Pre-LN 有什么代价，用什么办法补救？
6. 证明 $\text{RMSNorm}(\lambda x) = \text{RMSNorm}(x)$ ($\lambda > 0$)。(a) 这说明它是线性的还是非线性的？(b) 这个性质对权重初始化的尺度敏感性有什么影响？
7. (思考题) 如果把 LayerNorm 换成 BatchNorm (跨样本统计)，自回归语言模型会出什么问题？(提示：想想第 6 章的因果掩码在防什么。)

本章小结

- LayerNorm 是逐 token、沿特征维度的操作，不跨 token、不跨 batch。因此它不混合 token，与因果性相容。
- 第一步（减均值）= 右乘投影矩阵 $P = I - (1/d)\mathbf{1}\mathbf{1}^T$ ，把向量投到与全 1 方向垂直的 $(d-1)$ 维超平面上。 P 对称且幂等。
- 第二步（除标准差）= 缩放到半径 $\sqrt{d}$ 的球面上。输出范数恒为 $\sqrt{d}$ 。
- 两步合计丢弃两个自由度：整体平移和整体缩放。 γ 、 β 提供固定的替代值，不能恢复原始的 μ 和 σ 。
- RMSNorm 省掉第一步。之所以几乎不掉点，是因为高维中随机向量沿全 1 方向的分量本就约为 $1/\sqrt{d}$ ，第一步几乎无事可做。
- Pre-LN 只归一化读出来的副本，不动残差流本身，因此保留了第 8 章那条干净的梯度通路。这是现代模型的默认选择。
- 归一化是非线性的： $\text{RMSNorm}(\lambda x) = \text{RMSNorm}(x)$ ，对缩放完全免疫。

本章刻意没讲

投影矩阵的一般理论与正交分解、幂等矩阵的谱性质、归一化对优化景观影响的理论分析、各种归一化变体 (GroupNorm、ScaleNorm、DeepNorm) 的比较。

本章只需要一个投影矩阵和一个球面。这两个几何对象已经解释了 LayerNorm、RMSNorm 和 Pre-LN 的全部设计动机。

下一章预告

第9章出现了一个投影矩阵——它压缩空间、丢弃维度。

第10章要看它的反面：**正交矩阵**，一类什么都不丢的变换。它保持所有长度、所有夹角，只做旋转。

这类矩阵有一个漂亮的性质：两个旋转的复合还是旋转，而且角度相加。RoPE 位置编码整个建立在这一条上，正确性的证明只需要一行：

$$\langle R(m\theta)q, R(n\theta)k \rangle = q^\top R((n-m)\theta) k$$

绝对位置在内积里自动抵消，只剩相对位置。这是全书三个正式推导中的第二个，也是最短的一个。

第 10 章 正交、旋转与位置

本章要回答：绝对位置是怎么自动变成相对位置的？

读完本章你能做到：论证注意力天生看不见顺序；证明正交矩阵保持内积；用一行推导给出 RoPE 的正确性；解释长度外推方法在调什么。

本章含全书第二个正式推导（10.4 节），它只有三行，是三个推导中最短的。

10.1 困惑：注意力对顺序完全无感

先证明一件让人不安的事：不加位置编码的 Transformer，根本看不见词序。

把输入的 n 个 token 打乱。用第 2 章的框架看会发生什么：

- 所有的 Linear 层都是右乘 (W_q 、 W_k 、 W_v 、 W_1 、 W_2) —— 它们逐行独立作用，输入换个顺序，输出就跟着换个顺序，内容一个都没变。
- 打分是全对全的： $\text{scores}[i][j]$ 只取决于第 i 个和第 j 个 token 的内容，不问它们谁在前面。
- softmax 是逐行的，加权平均也是逐行的。

严格地说，若 P 是一个置换矩阵，则

$$\text{softmax}((PQ)(PK)^T)(PV) = \text{softmax}(P S P^T)(PV) = P \cdot A \cdot P^T P \cdot V = P (A V)$$

打乱输入 = 打乱输出，内容分毫不差。

"the cat sat" 和 "sat cat the" 在模型眼里是同一堆东西。顺序信息必须被显式地注入进去，否则它根本不存在。

这就是位置编码要解决的问题。本章讲目前的主流方案，而它的全部数学是一类特殊的矩阵。

10.2 正交矩阵：什么都不丢的变换

正交矩阵：满足 $R^T R = R R^T = I$ 的方阵。

它的关键性质是**保内积**。验证只需一步：

$$\langle Rx, Ry \rangle = (Rx)^T(Ry) = x^T R^T R y = x^T y = \langle x, y \rangle$$

内积保住了，长度和夹角自然也保住（第 5 章：范数和夹角都由内积定义）。

正交变换只旋转和翻转空间，不拉伸、不压缩、不丢弃任何东西。

和第 9 章形成完美对照：

	投影矩阵 P	正交矩阵 R
定义式	$P^2 = P$	$R^T R = I$
秩	$< d$ (不满)	d (满)
可逆	否	是，且 $R^{-1} = R^T$
对内积	改变	保持
几何	压扁到子空间	旋转整个空间

第 4 章说过线性变换一般不保长度和角度；正交矩阵是那个例外。这个例外，正好是位置编码需要的东西。

10.3 二维旋转与角度相加

最简单的正交矩阵是平面旋转：

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

两条性质是 RoPE 的全部代数基础，都可以用三角和角公式直接验证：

性质一（角度相加）： $R(\alpha) R(\beta) = R(\alpha + \beta)$

性质二（转置即反向）： $R(\theta)^T = R(-\theta)$

第二条其实就是正交性的具体形式： $R(\theta)^T R(\theta) = R(-\theta) R(\theta) = R(0) = I$ 。

记住这两条，下一节的推导就是三行。

10.4 正式推导：RoPE

想要什么

理想的位置编码应该让注意力分数只依赖**相对距离**。第 5 个词看第 2 个词，和第 105 个词看第 102 个词，如果内容相同，分数就该相同——它们的相对关系都是“往回三个位置”。

绝对位置编码（比如 GPT-2 那种在输入端加一个位置向量的做法）做不到这一点：两次的绝对位置不同，分数也就不同。

做法

RoPE 的方案是：**把位置信息编码成旋转角度**。

设 q 是位置 m 上的 query， k 是位置 n 上的 key。把它们各自旋转与自身位置成正比的角度：

$$q_m = R(m\theta) q \quad k_n = R(n\theta) k$$

（本节采用列向量写法，与文献一致。这里只涉及单个向量的旋转，不涉及批量数据，所以不用本书的行约定，以免公式变形。）

三行推导

$$\begin{aligned} \langle q_m, k_n \rangle &= (R(m\theta)q)^T (R(n\theta)k) \\ &= q^T R(m\theta)^T R(n\theta) k && \text{(转置分配)} \\ &= q^T R(-m\theta) R(n\theta) k && \text{(性质二)} \\ &= q^T R((n-m)\theta) k && \text{(性质一)} \end{aligned}$$

$$\langle q_m, k_n \rangle = q^T R((n-m)\theta) k$$

结果只含 $n - m$ 。绝对位置 m 和 n 在内积里自动抵消了，剩下的只有相对位置。

推导结束。全部工具就是 $R(\theta)^T = R(-\theta)$ 和 $R(\alpha)R(\beta) = R(\alpha+\beta)$ 。

推广到 d 维

真实向量是 768 维，不是 2 维。做法是**把 d 个坐标两两分成 $d/2$ 组**，每组是一个二维平面，各自独立旋转：

$$\text{第 } t \text{ 组的角度} = m \cdot \theta_t, \quad \theta_t = \text{base}^{(-2t/d)}, \quad \text{base 通常取 } 10000$$

整体的旋转矩阵是一个**块对角矩阵**，对角线上是 $d/2$ 个 2×2 旋转块。块对角的正交矩阵仍然是正交矩阵，所以上面的推导对每一组分别成立。

而总的内积是各组内积之和，每一项都只依赖 $n - m$ ，因此：

完整的 d 维注意力分数，只依赖相对位置。

10.5 为什么要用多个频率

θ_t 随组号递减。以 $d = 128$ 、 $\text{base} = 10000$ 为例：

组 t	θ_t	转一圈需要多少个位置
0（最高频）	1.0	约 6
32（中间）	0.01	约 630
63（最低频）	≈ 0.000116	约 54000

高频组分辨近距离，低频组分辨远距离。

类比钟表：秒针转得快，能区分几秒之差，但转一圈就重复了；时针转得慢，能区分几小时之差。几根不同速度的指针合起来，就能唯一地表示一个很大的时间范围。二进制计数也是同样的道理——不同位有不同的翻转频率。

$d/2$ 组不同频率的旋转合起来，让模型既能分辨"隔了 2 个词"和"隔了 3 个词"，也能分辨"隔了 2000 个词"和"隔了 3000 个词"。

10.6 长度外推：在调什么

训练时上下文长度是 4096，推理想用 32768，会发生什么？

低频组的角度跑出了训练时见过的范围。模型在训练中从未见过 $m \cdot \theta_t$ 超过某个值时的样子，行为无法预测，效果通常急剧下降。

主流的解决方案全都在动同一组数：

方法	做了什么
位置插值 (PI)	把位置 m 换成 m/k ，等于把所有 θ_t 缩小 k 倍，把长序列"压"回训练范围
NTK-aware	调大 base ，让低频转得更慢，高频基本不动
YaRN	按频率分段处理：高频不动、低频插值、中间过渡

它们都只是在调整 θ_t 的取值。一旦理解 10.5 节那张表，这些方法就不再是黑话，而是“该让哪几根指针转慢一点”的具体选择。

10.7 实现：不真的做矩阵乘法

块对角矩阵里全是零，真去做 768×768 的矩阵乘法极其浪费。实际实现只需要逐元素运算：

对第 t 组的两个坐标 (x_1, x_2) ：

$$\begin{aligned} x_1' &= x_1 \cdot \cos(m\theta_t) - x_2 \cdot \sin(m\theta_t) \\ x_2' &= x_1 \cdot \sin(m\theta_t) + x_2 \cdot \cos(m\theta_t) \end{aligned}$$

代价是 $O(d)$ 而不是 $O(d^2)$ 。在代码里通常写成一行：

```
q_rot = q * cos + rotate_half(q) * sin
```

其中 `rotate_half` 就是把每组的两个坐标交换并给一个取负号——把上面两行合并成向量形式的写法。

10.8 动手

```
import numpy as np

def rot(t):
    c, s = np.cos(t), np.sin(t)
    return np.array([[c, -s], [s, c]])

# 性质验证
print(np.allclose(rot(0.3) @ rot(0.7), rot(1.0)))    # True: 角度相加
print(np.allclose(rot(0.3).T, rot(-0.3)))           # True: 转置即反向
print(np.allclose(rot(0.3).T @ rot(0.3), np.eye(2))) # True: 正交

# 核心：内积只依赖相对位置
q, k, th = np.array([1.0, 2.0]), np.array([0.5, -1.0]), 0.3
for m, n in [(2, 5), (10, 13), (100, 103), (5000, 5003)]:
    print(m, n, round((rot(m*th) @ q) @ (rot(n*th) @ k), 12))
```

四行输出的数值完全相同——四对位置相隔都是 3。

完整的 d 维版本：

```
def rope(x, pos, base=10000.0):
    d = x.shape[-1]
    th = pos * base ** (-np.arange(0, d, 2) / d)      # (d/2,) 各组角度
    c, s = np.cos(th), np.sin(th)
    x1, x2 = x[0::2], x[1::2]
    out = np.empty_like(x)
    out[0::2], out[1::2] = x1*c - x2*s, x1*s + x2*c
    return out

rng = np.random.default_rng(0)
q, k = rng.normal(size=64), rng.normal(size=64)
print(rope(q, 3) @ rope(k, 7))          # 相对距离 4
print(rope(q, 500) @ rope(k, 504))     # 相对距离 4 —— 数值相同
```

建议做的三个实验：

1. **看见相对性（本章必做）**。运行上面第二段，把 (3, 7) 和 (500, 504) 换成任意相隔 4 的位置对，确认内积恒定。然后改成相隔 5，看数值怎么变。
2. **画出衰减曲线**。固定 $q = k$ （同一个随机向量），令相对距离 Δ 从 0 变到 2000，画出 `rope(q,0) @ rope(q, $\Delta$ )`。你会看到内积随距离震荡衰减——**RoPE 自带一种“距离越远越不相关”的先验**，这是它效果好的原因之一。
3. **看见外推失效**。把 base 从 10000 改成 100，重画上面的曲线。低频组转得快多了，远距离迅速陷入混乱。再试 base = 1000000，观察相反的效果。**这个实验能让你直观理解 NTK-aware 那类方法在调什么。**

10.9 落回 Transformer

```
q = apply_rope(q, positions)    # 只作用于 q
k = apply_rope(k, positions)    # 只作用于 k
scores = q @ k.transpose(-2, -1) / math.sqrt(d_k)
out = attn @ v                  # v 没有被旋转
```

注意 v 没有被旋转。这是刻意的。

用第 6 章的分工来理解：位置信息应该影响“**谁该关注谁**”（打分阶段），不应该影响“**搬运什么内容**”（加权平均阶段）。 q 和 k 决定前者， v 决定后者。

如果给 v 也加上旋转，同一个词的 value 会因为所处位置不同而改变，搬运的内容就被位置污染了。

另一个细节：RoPE 在**每一层**都要重新施加一次，而不是像绝对位置编码那样只在输入端加一次。因为它作用的是每层新算出来的 q 和 k ，不是残差流本身。这也意味着位置信息不会占用第 8 章讲的残差流带宽——它不写进总线，只在读取时临时叠加。

10.10 自测

1. 证明正交矩阵保持内积。据此说明它也保持长度和夹角。（提示：第 5 章说长度和夹角都由内积定义。）
2. 用三角和角公式证明 $R(\alpha)R(\beta) = R(\alpha+\beta)$ ，并由此推出 $R(\theta)^T = R(-\theta)$ 。
3. 完整重写 RoPE 的三行推导。明确指出每一步用到了哪条性质。
4. $d = 128$, $\text{base} = 10000$ 。（a）最高频组和最低频组的 θ 各是多少？（b）各自转一圈需要多少个位置？（c）如果只用单一频率 $\theta = 1$ ，会出什么问题？
5. 位置插值把 m 换成 m/k 。（a）相对位置差 $(n-m)$ 变成了什么？（b）为什么这能让模型处理更长的序列？（c）代价是什么？（提示：想想相邻位置的角度差变小了。）
6. 为什么 RoPE 只作用于 q 和 k ，不作用于 v ？如果给 v 也加上旋转，会破坏什么？
7. （a）证明置换输入会置换输出（10.1 节）。（b）如果彻底去掉位置编码，模型还能学到什么？它会退化成什么模型？
8. （思考题）第 9 章的投影矩阵 P 和本章的正交矩阵 R 是两个极端。有没有既不丢维度、又改变长度的线性变换？举一个最简单的例子。

本章小结

- **注意力天生对顺序无感**：打乱输入只会打乱输出，内容不变。位置信息必须显式注入。
- **正交矩阵 $R^S R = I$ 保持内积**，因而保持长度和夹角。它与第 9 章的投影矩阵形成完美对照：一个什么都不丢，一个专门丢东西。
- 二维旋转的两条性质是全部基础： **$R(\alpha)R(\beta) = R(\alpha+\beta)$** 和 **$R(\theta)^S = R(-\theta)$** 。
- **RoPE 恒等式**： $\langle R(m\theta)q, R(n\theta)k \rangle = q^T R((n-m)\theta)k$ 。绝对位置在内积里自动抵消，只剩相对位置。
- d 维实现是块对角旋转， **$d/2$ 组不同频率**：高频分辨近距离，低频分辨远距离，如同钟表的多根指针。
- 长度外推方法（PI、NTK-aware、YaRN）本质上都在**调整 θ_t** 。
- 实现上只需 $O(d)$ 的逐元素运算，不做真正的矩阵乘法。
- **RoPE 只作用于 q 和 k ，不作用于 v** ：位置该影响"关注谁"，不该影响"搬运什么"。它每层重新施加，不占用残差流带宽。

本章刻意没讲

正交群与特殊正交群的结构、行列式与反射的区分、旋转的复数与四元数表示、Givens 旋转与 QR 分解、正弦位置编码与 RoPE 的理论关联、各类外推方法的完整推导。

本章只需要一个 2×2 旋转矩阵和它的两条性质。RoPE 的正确性由此三行可得，其余属于可选的背景知识。

第三部结束

三章看下来，模型的几何长相已经清楚了：

- 第 8 章：一条 768 维的加法总线，各层往里读写；高维的慷慨让它装下远超 768 个概念。
- 第 9 章：读取总线时做一次投影加缩放，把向量校准到球面上。
- 第 10 章：打分前对 q 、 k 施加旋转，让绝对位置在内积中抵消成相对位置。

投影、加法、旋转——三种最基本的几何操作，撑起了整个架构。

下一章预告

还剩一个大部件没拆：FFN。它占了模型三分之二的参数，却几乎没有被前十章讨论过。

第 11 章会回答两个问题。第一，为什么要先升到 4d 维再降回来——这个"胖中间层"在做什么？第二，也是从第 4 章拖到现在的那个问题：**非线性究竟带来了什么？**

你会看到第 2 章的线性组合视角最后一次登场，把 FFN 变成一组"键值记忆"： W_1 的行负责匹配， W_2 的行负责写入。而第 8 章的读写图景会在这里完全闭合。

第 11 章 升维、非线性与深度

本章要回答：FFN 占了模型三分之二的参数，它到底在做什么？非线性究竟带来了什么？

读完本章你能做到：把 FFN 读成一组键值记忆；用一个对角矩阵精确说明非线性带来的是什么；讲清注意力与 FFN 的完整分工。

本章兑现第 4 章拖到现在的那个问题。第 2 章的三种读法会在这里最后一次全部登场。

11.1 困惑：最简单的部件，最多的参数

FFN 的全部内容是两个矩阵夹一个激活函数：

$$\text{FFN}(x) = \sigma(x W_1) W_2 \quad W_1 \in \mathbb{R}^{d \times 4d}, W_2 \in \mathbb{R}^{4d \times d}$$

比注意力简单得多——没有打分，没有 softmax，没有跨 token 的交互。

但把参数量算一算：

组件	矩阵	参数量
注意力	W_q, W_k, W_v, W_o , 各 $d \times d$	$4d^2$
FFN	$W_1 (d \times 4d) + W_2 (4d \times d)$	$8d^2$

FFN 占了每层参数的三分之二。

一个如此简单的结构，凭什么值这么多参数？本章的答案是：它是模型存放知识的地方，而知识需要空间。

11.2 第一步：W1 的列是 4d 把钥匙

用第 2 章的点积视角看 $x @ W_1$ 的第 k 个输出分量：

$$(x W_1)_k = x \cdot (W_1 \text{ 的第 } k \text{ 列})$$

每一列是一个方向，输出的每一个分量就是"x 和这个方向有多对齐"。

W_1 有 $4d$ 列 = $4d$ 个并行的检测器。每一列问一个问题: "输入里有没有这个模式?"

这正是第 8 章"读 = 右乘矩阵"的具体形态。注意力头只读几十个方向 ($d_k = 64$) , 而 FFN 一口气读 $4d = 3072$ 个方向。

11.3 第二步: W_2 的行是 $4d$ 个写入方向

用第 2 章的线性组合视角看 $h @ W_2$:

$$\text{out} = \sum_k h_k \cdot (W_2 \text{ 的第 } k \text{ 行})$$

每一行是一个 d 维向量——一个候选的写入方向。激活值 h_k 决定它被写入多少。

把两步合起来:

FFN 是一组键值记忆

- W_1 的第 k 列 = 第 k 条记忆的键 (什么情况下触发)
- W_2 的第 k 行 = 第 k 条记忆的值 (触发后往残差流写什么)
- 激活值 h_k = 匹配强度

每次前向传播: $4d$ 条记忆同时被检索, 被激活的那些把自己的值加权写回残差流。

与注意力的对照

这个图景让 Transformer 的两个部件变成同一件事的两个版本:

	键从哪来	值从哪来	有多少条
注意力	当前上下文的 token	当前上下文的 token	n 条 (随输入变化)
FFN	权重 W_1 的列	权重 W_2 的行	$4d$ 条 (固定, 训练时学到)

注意力从上下文取信息, FFN 从参数取信息。

这解释了一个重要的经验事实: 模型记住的事实性知识, 主要存放在 FFN 里。"巴黎是法国的首都"这条信息不在任何上下文中, 它必须被存在权重里——而 FFN 正是那个键值存储。模型编辑类的工作 (定位并修改某条事实) 几乎都在改 FFN 的权重, 原因就在这里。

11.4 非线性带来了什么：一个精确的答案

第 4 章证明了没有非线性时深度毫无意义，但没说非线性带来了什么。现在可以给一个精确的回答。

取 $\sigma = \text{ReLU}$ （最容易分析，结论对其他激活函数类似）。ReLU 只做一件事：小于 0 的置零，大于 0 的原样通过。

把"哪些通过"记成一个对角矩阵 D_x ：对角线上第 k 个元素，若 $h_k > 0$ 就是 1，否则是 0。于是

$$\text{FFN}(x) = x W_1 D_x W_2$$

看这个式子。 W_1 和 W_2 是固定的，唯一随输入变化的是中间那个 D_x 。

所以：

非线性带来的，是"根据输入选择用哪个线性映射"的能力。

没有 σ 时， D 恒等于单位矩阵，模型只有一个线性映射 $W_1 W_2$ 。有了 ReLU， D_x 有 2^{4d} 种可能的取值—— $d = 768$ 时是 2^{3072} 种。

模型不再是一个线性变换，而是一个巨大的线性变换库，外加一套依输入选择的规则。

用键值记忆的语言说同一件事： σ 是门控。它决定这一步激活哪些记忆条目。没有它，所有条目永远全部激活、按固定权重求和，"检索"就不存在了。

分段线性与深度

由此还能看出深度为什么有用。每一个 D_x 对应输入空间中的一块区域，区域内部是线性的，跨区域时映射发生切换。整个网络是一个分段线性函数，区域数决定了它能刻画多复杂的形状。

关键在于：叠加 L 层时，区域数按层数指数增长，而参数量只是线性增长。这就是"深度比宽度更划算"的具体机制——也是第 4 章那句"没有非线性时深度贡献严格为零"的正面回答。

顺带：升维在纯线性下完全白费

$W_1 W_2$ 的腰是 $4d = 3072$ ，大于 $d = 768$ ，所以 $\text{rank}(W_1 W_2)$ 可以达到满秩 768，没有任何秩损失。

这一点值得注意：升维不像第 4 章那个降维例子那样会损失秩。但它仍然毫无意义——因为无论秩多少，结果仍然只是一个矩阵。

升维的价值完全依赖于中间那个非线性。把 σ 拿掉，那 $8d^2$ 个参数瞬间坍缩成 d^2 个的效果。

11.5 为什么是 4 倍

两个理由，一个数字。

理由一：容量。 $4d = 3072$ 条记忆槽。听起来很多，但相对于一个语言模型需要记住的事实、习语、语法规则，仍然远远不够。

于是 FFN 内部也在用第 8 章的叠加：**特征数远超槽位数，以近似正交方向叠加存储。**事实上，"多义神经元"最早就是在 FFN 的中间层被观察到的——你去看某个中间维度什么时候激活，会得到一份毫无关联的清单。原因第 8 章已经讲过：特征方向不与坐标轴对齐。

理由二：高维更容易切开。 逐元素的 ReLU 只能沿坐标轴切分空间。维度越高，可用的切分面越多，能刻画区域形状就越精细。**先升维再用非线性，比直接在 d 维里用非线性能表达更多东西。**

至于为什么恰好是 4——它是经验值，不是推导出来的。原始 Transformer 论文选了这个比例，后续工作大多沿用。现代模型改用 SwiGLU 之后，中间维通常取约 $8d/3$ ，为的是在多一个矩阵的情况下把总参数量维持在 $8d^2$ 附近。

(SwiGLU 的形式是 $(x \cdot W_{\text{gate}} \odot \sigma(x \cdot W_{\text{up}})) \cdot W_{\text{down}}$ ，用一个显式的门控分支替代单一激活函数。它比 ReLU/GELU 好一些，但改变的是门控的形式，不是本章的图景——键、值、门控三者的角色完全不变。)

11.6 闭合：注意力与 FFN 的完整分工

到这里，第 2 章那句"左乘混 token，右乘混特征"可以展开成完整的对照：

	注意力	FFN
矩阵乘法方向	唯一的 左乘 ($\text{attn} @ V$)	全是 右乘
跨 token	是 (唯一的通信通道)	否 (逐 token 独立)
信息来源	当前上下文	模型参数
键值来自	输入 (动态)	权重 (静态)
参数量	$4d^2$	$8d^2$
计算复杂度	$O(n^2d)$	$O(nd^2)$
长序列时的瓶颈	是	否

一层 Transformer = 一次通信 + 一次计算。

注意力把需要的信息搬到当前位置，FFN 用参数里的知识加工它。堆 L 层，就是交替进行 L 次搬运与加工。

这是全书最高层的图景。前十章的所有细节，都可以挂在这两句话下面。

11.7 参数量：把账算平

每层：

$$\text{注意力 } 4d^2 + \text{FFN } 8d^2 = 12d^2$$

以 GPT-2 small 验算（ $d = 768$, $L = 12$, $V = 50257$ ）：

部分	计算	参数量
12 层主体	$12 \times 12 \times 768^2$	84,934,656
词嵌入	50257×768	38,597,376
位置嵌入	1024×768	786,432
合计		$\approx 124.3 \text{ M}$

对得上官方的 124M。（归一化层的 γ 、 β 共约 4 万，可忽略。）

这个 $12d^2$ 是估算模型规模最有用的一个数字。第 12 章会用它来做几道估算题。

11.8 动手

```
import numpy as np
rng = np.random.default_rng(0)
d, d_ff = 768, 3072
x = rng.normal(size=d)
W1 = rng.normal(size=(d, d_ff)) / np.sqrt(d)
W2 = rng.normal(size=(d_ff, d)) / np.sqrt(d_ff)

h = np.maximum(x @ W1, 0)          # ReLU 门控
out = h @ W2
print((h > 0).mean())              # ≈ 0.5: 一半记忆槽被激活

# 读法二：写入是 W2 各行的加权和
print(np.allclose(out, sum(h[k] * W2[k] for k in range(d_ff))))    # True

# 非线性 = 一个依输入而变的对角门控矩阵
D = np.diag((x @ W1 > 0).astype(float))
print(np.allclose(out, x @ W1 @ D @ W2))    # True ← 本章的核心式子

# 去掉非线性：坍塌成一个矩阵，且秩没有任何损失
print(np.allclose(x @ W1 @ W2, x @ (W1 @ W2)),
      np.linalg.matrix_rank(W1 @ W2))    # True 768 (满秩，但仍是一个矩阵)
```

建议做的三个实验：

1. **看见门控矩阵（本章必做）。** 上面第三段确认 $\text{FFN}(x) = x W_1 D_x W_2$ 之后，换一个输入 x_2 ，打印两个 D 的对角线有多少位不同（ $(D1 \neq D2).sum()$ ）。不同的输入，用的是不同的线性映射——这就是非线性的全部内容。
2. **数一数可能的映射数。** 对 100 个随机输入分别求 D ，统计有多少个互不相同（几乎必然是 100 个）。然后想清楚： 2^{3072} 这个数和宇宙原子数比是什么量级？
3. **测量激活稀疏度。** 上面显示随机初始化时约 50% 的槽被激活。训练好的模型远比这稀疏（常见是百分之几）。想一想：稀疏激活和第 8 章的叠加有什么关系？如果 90% 的槽同时激活，键值检索还成立吗？

11.9 落回 Transformer

```
class MLP(nn.Module):
    def __init__(self, d):
        self.fc = nn.Linear(d, 4*d)    # W1: 4d 把钥匙
        self.proj = nn.Linear(4*d, d)  # W2: 4d 个写入方向
    def forward(self, x):
        return self.proj(gelu(self.fc(x))) # 检索 → 门控 → 写入
```

三行代码，三个角色。这是整个模型里参数最多的部件，而它的全部结构就在这三行里。

注意 FFN 完全不知道其他 token 的存在——它对序列里每个位置独立计算，可以完全并行。所以尽管它占三分之二的参数，在长序列场景下它不是瓶颈： $O(nd^2)$ 是线性的，而注意力的 $O(n^2d)$ 才是那个会爆炸的项。

11.10 自测

1. (a) W_1 的列扮演什么角色？用第 2 章哪种读法看出来的？(b) W_2 的行扮演什么角色？用的是哪种读法？
2. 证明 $\text{FFN}(x) = x W_1 D_x W_2$ ，其中 D_x 是对角 0/1 矩阵。(a) D_x 有多少种可能取值？(b) 用一句话说明"非线性带来了什么"。(c) 如果去掉 ReLU， D 变成什么？
3. 去掉激活函数后：(a) FFN 退化成什么？(b) $\text{rank}(W_1 W_2)$ 的上界是多少，会有损失吗？(c) 既然秩没损失，为什么说"升维完全白费"？
4. 某模型 $d = 4096$ ， $L = 32$ 。(a) 每层参数量？(b) 主体总参数量？(c) FFN 占多少比例？(d) 若把 FFN 中间维从 $4d$ 改成 $2d$ ，总参数量变成多少？
5. 复述注意力与 FFN 的分工，至少覆盖：跨不跨 token、信息来源、参数量、计算复杂度、长序列瓶颈。
6. 为什么模型的事实性知识主要存放在 FFN 而非注意力里？（提示：注意力的键值来自哪里？）
7. (思考题) FFN 有 $4d = 3072$ 个中间维，但模型需要表示的特征远不止 3072 个。(a) 这在第 8 章的语言里叫什么？(b) 它对"某个中间维代表什么概念"这类分析意味着什么？(c) 稀疏激活为什么是这套机制成立的前提？

本章小结

- FFN 占每层参数的三分之二 ($8d^2$ vs 注意力的 $4d^2$)，每层合计 $12d^2$ 。
- W_1 的列 = $4d$ 把钥匙（点积视角）； W_2 的行 = $4d$ 个写入方向（线性组合视角）。FFN 是一组键值记忆。
- 注意力从上下文取信息，FFN 从参数取信息。这解释了事实知识为何主要存在 FFN 里。
- $\text{FFN}(x) = x W_1 D_x W_2$ ，其中 D_x 是依输入而定的对角门控矩阵。非线性带来的正是"根据输入选择线性映射"的能力——从 1 个映射变成 2^{4d} 个。
- 网络因此是分段线性函数，区域数随层数指数增长，这是深度划算的机制。
- 升维不损失秩，但没有非线性时仍然坍缩成一个矩阵——升维的价值完全依赖于 σ 。
- 4 倍是经验值。FFN 内部同样存在叠加，多义神经元最早就在这里被发现。
- 一层 = 一次通信（注意力）+ 一次计算（FFN）。

本章刻意没讲

万能逼近定理、ReLU 网络区域数的精确计数、激活函数的性质比较（GELU 与 SwiGLU 的推导）、专家混合（MoE）对 FFN 的稀疏化改造、知识编辑方法的具体算法。

本章只需要一个对角门控矩阵。它已经精确回答了"非线性带来了什么"，其余属于可选的扩展阅读。

下一章预告

十一章过去，每个部件都拆开看过了。

第 12 章要把它们拼回去：从零手写一层完整的 Transformer，只用 NumPy，不用任何框架。张量、广播、einsum 会在那里作为工具出现——你此时已经有足够的形状直觉，不需要为它们单独开一章。

那一章还会用 $12d^2$ 做几道规模估算：给定参数量反推模型形状、估算显存与推理成本、判断一块显卡能跑多大的模型。

写完那一层代码，这本书的任务就完成了。你会发现自己在读它的时候，脑子里同时挂着四样东西：**每个张量的形状、每个矩阵乘法属于哪种读法、每个组件在残差流上读写什么、以及每个设计选择背后的那条线性代数理由。**

第 12 章 从零拼出一层

本章要回答：前十一章能拼成什么？

读完本章你能做到：用四十行 NumPy 写出一层完整的 Transformer；用三个测试验证它是对的；用 $12d^2$ 估算任意模型的参数量、显存和算力需求。

本章不引入任何新的线性代数。它只是把前十一章的东西拼起来，然后看看拼出来的是什么。

12.1 目标

我们要写的是一层现代 Transformer：Pre-Norm + RMSNorm + RoPE + 因果多头注意力 + FFN。不用 PyTorch，只用 NumPy——框架会隐藏形状，而形状恰恰是这本书训练的东西。

写之前先把地图列出来。每一块对应哪一章：

组件	章节	核心内容
RMSNorm	第 9 章	缩放到半径 $\sqrt{d}$ 的球面
三次投影	第 4 章	每个 token 独立降维
拆头 / 拼头	第 3 章	reshape + transpose
RoPE	第 10 章	二维旋转，只作用于 q、k
打分 $\div \sqrt{d_k}$	第 5 章	方差校准
因果掩码	第 6 章	先屏蔽，后归一化
attn @ V	第 2、6 章	唯一的左乘，凸组合
W_o	第 7 章	h 个贡献之和
FFN	第 11 章	键值记忆 + 门控
两个 $x = x + \dots$	第 8 章	残差流

12.2 逐块实现

归一化（第 9 章）

```
def rms_norm(x, g, eps=1e-6):
    # x: (n, d)。逐 token 沿特征维度，不跨 token
    return g * x / np.sqrt((x**2).mean(-1, keepdims=True) + eps)
```

一行。缩放到球面，`g` 是可学习的尺度。

旋转位置编码（第 10 章）

```
def rope(x, base=10000.0):
    # x: (h, n, dk)
    h, n, dk = x.shape
    inv = base ** (-np.arange(0, dk, 2) / dk)      # (dk/2,) 各组频率
    th = np.arange(n)[::2, None] * inv           # (n, dk/2) 角度 = 位置 × 频率
    c, s = np.cos(th), np.sin(th)
    x1, x2 = x[..., 0::2], x[..., 1::2]
    out = np.empty_like(x)
    out[..., 0::2] = x1 * c - x2 * s
    out[..., 1::2] = x1 * s + x2 * c
    return out
```

注意这里没有做任何矩阵乘法——块对角旋转只需逐元素运算， $O(d)$ 而非 $O(d^2)$ 。

softmax（第 5、6 章）

```
def softmax(z, axis=-1):
    z = z - z.max(axis, keepdims=True)           # 数值稳定
    e = np.exp(z)
    return e / e.sum(axis, keepdims=True)
```

注意力（第 2~7、10 章）

```
def attention(x, Wq, Wk, Wv, Wo, h):
    n, d = x.shape
    dk = d // h
    # 第 4 章：三次投影 | 第 3 章：拆头，让 h 变成 batch 维
    q, k, v = [(x @ W).reshape(n, h, dk).transpose(1, 0, 2) for W in (Wq, Wk, Wv)]
    q, k = rope(q), rope(k)                      # 第 10 章：v 不旋转
    s = np.einsum('hqd,hkd->hqk', q, k) / np.sqrt(dk)  # 第 5 章
    s = s + np.triu(np.full((n, n), -1e9), 1)         # 第 6 章：先屏蔽
    a = softmax(s)                                  # 第 6 章：后归一化
    o = np.einsum('hqk,hkd->hqk', a, v)             # 第 2、6 章：唯一的左乘
    o = o.transpose(1, 0, 2).reshape(n, d)          # 第 3 章：拼头（顺序不能颠倒）
    return o @ Wo, a                               # 第 7 章：h 个贡献之和
```

这九行是整本书的重心。每一行都能追溯到某一章的一个结论。

FFN（第 11 章）

```
def ffn(x, W1, W2):
    return np.maximum(x @ W1, 0) @ W2          # 检索 → 门控 → 写回
```

组装（第 8 章）

```
def block(x, p, h):
    x = x + attention(rms_norm(x, p['g1']), p['Wq'], p['Wk'], p['Wv'], p['Wo'], h)[0]
    x = x + ffn(rms_norm(x, p['g2']), p['W1'], p['W2'])
    return x
```

两个加号就是整个模型的主干。归一化的是读出来的副本，残差流本身从不被归一化——这就是 Pre-LN。

跑起来

```
import numpy as np
rng = np.random.default_rng(0)
n, d, h = 16, 64, 4
sc = lambda a, b: rng.normal(size=(a, b)) / np.sqrt(a)
p = dict(g1=np.ones(d), g2=np.ones(d),
          Wq=sc(d, d), Wk=sc(d, d), Wv=sc(d, d), Wo=sc(d, d),
          W1=sc(d, 4*d), W2=sc(4*d, d))

x = rng.normal(size=(n, d))
y = block(x, p, h)
print(x.shape, y.shape)          # (16, 64) (16, 64) ← 同形，可以无限堆叠
```

全部代码不到四十行。这就是一层 Transformer 的全部。

12.3 三个验证

写完不算完。下面三个测试，每一个都对应本书的一个结论。

测试一：权重确实是凸组合（第 6 章）

```
_, a = attention(rms_norm(x, p['g1']), p['Wq'], p['Wk'], p['Wv'], p['Wo'], h)
print(np.allclose(a.sum(-1), 1.0))          # True: 每行和为 1
print(a.min() >= 0)                         # True: 非负
print(np.abs(np.triu(a, 1)).max())          # 0.0: 上三角严格为零
```

前两条保证输出落在 value 的凸包内；第三条保证没有看未来。

测试二：因果性（第 6 章）

这是最有说服力的一个测试。改动第 10 个位置的输入，前 10 个位置的输出应当分毫不动：

```
x2 = x.copy()
x2[10] += 5.0
y2 = block(x2, p, h)
print(np.abs(y - y2)[:10].max())      # ≈ 0      前面完全不受影响
print(np.abs(y - y2)[10:].max())      # 明显 > 0 后面变了
```

如果第一个数不是 0，说明掩码写错了——而这类 bug 不会报错（第 3 章）。这个测试是抓住它的唯一办法。

测试三：RoPE 只编码相对位置（第 10 章）

```
q = rng.normal(size=(1, 64, 8))
r = rope(q)
print(np.allclose(r[0, 3] @ r[0, 7], r[0, 50] @ r[0, 54])) # True: 相距都是 4
```

建议再做一个实验：把 attention 里的 `s + mask` 改成"先 softmax 再把上三角置零"，重跑测试一。你会看到每行和不再是 1——第 6 章那个论证的实物版本。

12.4 规模估算：12d² 的用法

第 11 章算出每层 12d²。这个数字可以支撑一整套工程估算。

参数量

$$N \approx 12 \cdot L \cdot d^2 \quad (\text{主体}) \quad + \quad V \cdot d \quad (\text{嵌入, 权重绑定时算一次})$$

验算 Llama-2-7B ($d = 4096$, $L = 32$, $V = 32000$) :

```
主体: 12 × 32 × 40962 = 6.44 G
嵌入: 32000 × 4096 × 2 = 0.26 G
合计: 约 6.7 G ← 官方 6.74 G
```

（现代模型用 SwiGLU 和 GQA 会让实际值偏离几个百分点，但 12d² 作为估算足够好。）

算力

两条经验公式，够用于绝大多数估算：

	每个 token 的 FLOPs
推理（前向）	$\approx 2N$
训练（前向 + 反向）	$\approx 6N$

来源很简单：每个参数在前向中参与一次乘和一次加，即 2 FLOPs；反向传播的代价约为前向的两倍，合计 3 倍。

显存

三部分：

权重 = $N \times$ 每参数字节数 （fp16 是 2 字节，int4 是 0.5 字节）
KV 缓存 = $2 \times L \times n \times d \times$ 字节数 （K 和 V 各存一份）
激活值 = 视实现而定，推理时较小

KV 缓存值得单独算一次。Llama-2-7B，序列 4096，fp16：

$2 \times 32 \times 4096 \times 4096 \times 2 \text{ 字节} \approx 2.1 \text{ GB}$

一条序列就要 2 GB。 而权重只要 14 GB。这解释了为什么长上下文推理的瓶颈往往不是模型大小，而是 KV 缓存——它随序列长度和 batch 大小**线性增长**，很快就会超过权重本身。

（GQA、MQA 这类技术的全部目的就是缩小这个数字：让多个 query 头共享同一组 K、V。）

三道估算题

1. 某模型 $N = 70 \text{ G}$ ， $d = 8192$ 。用 $12Ld^2$ 反推 L 大约是多少？（实际是 80，你的估算会偏大一些，想想为什么。）
2. 用 7B 模型处理 32768 长度的序列，KV 缓存要多少 GB？一块 24 GB 显卡在 fp16 下还剩多少空间给权重？如果权重量化到 int4 呢？
3. 用 1 万亿 token 训练一个 7B 模型，总共需要多少 FLOPs？如果 1000 张卡各自实际发挥 150 TFLOP/s，需要多少天？

12.5 落回 Transformer：完整的一层

最后再看一遍那两行主干，这次带上全部注解：

```
x = x + attn(norm(x))    # 通信：把需要的信息从别的位置搬过来
x = x + mlp(norm(x))     # 计算：用参数里的知识加工它
```

- **norm** 把读出的副本校准到球面（第 9 章）
- **attn** 是模型里唯一跨 token 的操作，输出是 value 的凸组合（第 2、6 章）
- **mlp** 逐 token 独立，从 4d 条记忆里检索并写回（第 11 章）
- 两个 **+** 让贡献可累加、梯度可直达、消融可定义（第 8 章）

堆 L 次，前面接一个嵌入表（第 1 章），后面接一个 unembedding，就是一个完整的语言模型。

12.6 总复习

导读里那八道题，现在应该都能答了：

1. 不查资料写出 $d=512$ 、 $h=8$ 的一层里每个张量的形状。
2. 用“行向量加权求和”解释 $A \cdot V$ ，说明输出为何落在 V 的凸包内。
3. 从方差角度推导 $\sqrt{d_k}$ 。
4. 说明 $W_q W_k^T$ 的秩上界，以及这对多头设计意味着什么。
5. 证明 $\langle R(m\theta)_q, R(n\theta)_k \rangle$ 只依赖 $n-m$ 。
6. 说明 LayerNorm 把向量映到了什么几何对象上。
7. 用 $12d^2$ 估算 GPT-2 的参数数量。
8. 解释为什么去掉所有非线性后，96 层模型等价于 1 层。

再加两道本书后半程的：

1. 论证 768 维空间为什么能容纳远超 768 个概念，给出定量依据。
2. 写出 $\text{FFN}(x) = x W_1 D_x W_2$ ，并用它说明非线性带来了什么。

12.7 结语：你现在会什么

回头看，这本书只教了很少的东西：

- **一个对象**：向量，以及一行一个 token 的矩阵。
- **一个运算**：矩阵乘法，和它的三种读法。
- **三种几何操作**：投影（压扁）、加法（叠加）、旋转（保持）。
- **两个不等式**： $\text{rank}(AB) \leq \text{腰}$ ；凸组合的范数不超过最长的那个。
- **两个数字**： $1/\sqrt{d}$ 和 $\sqrt{\ln N}$ 。

就这些。行列式、矩阵求逆、高斯消元、特征分解、若尔当标准型——一个都没用上。**这不是因为它们不重要，而是因为它们服务的是别的问题。**

真正的收获不是记住了这些条目，而是养成了一种阅读方式。现在你看一行 Transformer 代码时，脑子里会同时挂着四样东西：

每个张量的形状是什么；这个矩阵乘法属于哪种读法；这个组件在残差流上读什么、写什么；这个设计选择背后是哪条线性代数理由。

这四条一旦并行运转，模型就不再是一个黑箱，而是一串可以逐行解释的选择。

往下走

如果你想理解训练：去看附录 A 的谱视角。SVD、谱范数、条件数——它们回答的是“梯度为什么爆炸”“学习率该怎么设”“初始化的尺度从哪来”。那是另一条主线，和本书正交。

如果你想理解模型内部：第 8 章的叠加是可解释性研究的入口。稀疏自编码器、特征方向、电路分析，用到的线性代数不超过本书的范围，只是问题换了。

如果你想补齐通识：附录 B 列了本书删掉的内容，以及在哪些场景下你会需要它们。不必着急——需要时再学，成本很低，而且那时你会带着具体问题去学，效率高得多。

如果你想验证自己：找一个真实模型的实现（nanoGPT 只有三百行），逐行读一遍。你会发现绝大部分内容你都见过，剩下的是工程细节而非数学。

本章小结

- 一层完整的 Transformer 可以在**四十行 NumPy** 内写出来，每一行都能追溯到本书的某一章。
- 三个验证：权重和为 1 且非负（凸组合）、上三角为零（因果掩码）、改动位置 t 不影响前 $t-1$ 个输出（因果性）。**最后一个是抓掩码 bug 的唯一可靠办法。**
- $N \approx 12Ld^2 + Vd$ ；推理每 token 约 $2N$ FLOPs，训练约 $6N$ 。
- **KV 缓存 = $2 \cdot L \cdot n \cdot d$ 字节数**，随序列长度线性增长，长上下文场景下常常比权重更吃显存。
- 全书用到的线性代数：一个对象、一个运算的三种读法、三种几何操作、两个不等式、两个数字。

全书完

第 1 章开头问的是：一个词怎么变成一串数字。

十二章之后，答案是：它变成一个 768 维向量，被扔进一条贯穿全模型的加法总线；每一层从总线上读走它关心的几十个方向，和别的位置比一比、从参数里检索一些知识，再把结果加回去；最后用同一张嵌入表读出来，变成下一个词的概率。

中间没有任何一步超出这本书讲过的东西。